01 Introduction to Ensemble Learning



Definition and Core Concepts

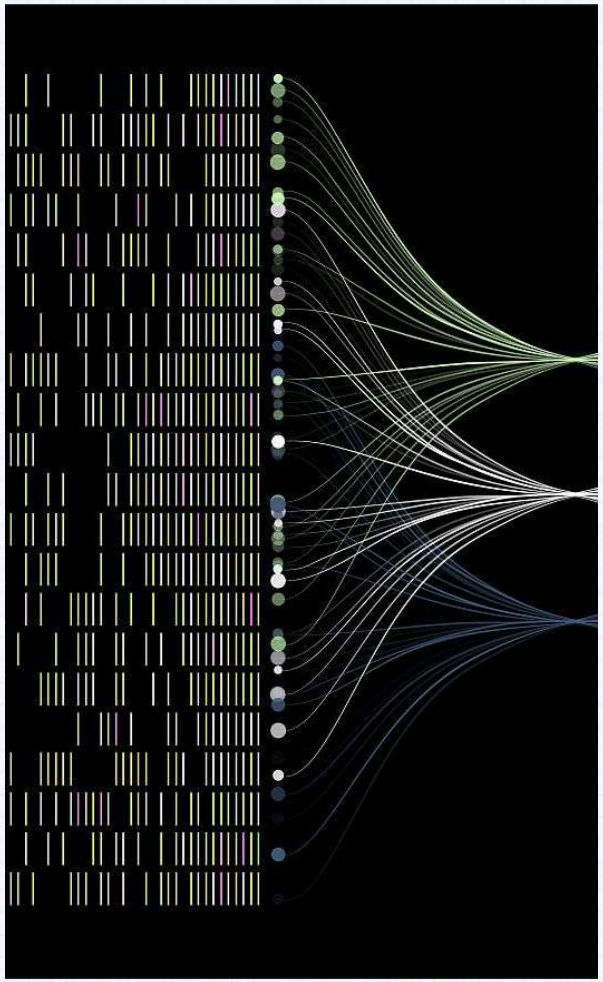
- Combination model: Ensemble learning improves overall performance by combining the prediction results of multiple base learners, with the core idea of "collective intelligence is better than individuals". Typical methods include Bagging, Boosting, and Stacking.

- Deviation variance trade-off: Ensemble methods can effectively balance the bias and variance of the model, Bagging reduces variance, Boosting reduces bias, and thus improves generalization ability.

- Diversity requirement: Base learners need to have diversity (such as different algorithms, data subsets, or feature spaces) to avoid collective errors, which is a key prerequisite for ensemble performance.

- Parallel and serial training: Bagging methods (such as random forest) can train base models in parallel, while Boosting methods (such as AdaBoost) require serial iteration to adjust sample weights.

Benefits of Ensemble Methods



Robustness enhancement

By aggregating the outputs of multiple models, ensemble methods significantly reduce sensitivity to noisy data and overfitting, especially suitable for high variance scenarios.

Prediction accuracy improvement

empirical research shows that the integrated model is often better than the single model in classification and regression tasks, such as XGBoost's dominant performance in the Kaggle contest.

Flexibility and compatibility

Supports heterogeneous model integration (such as combining decision trees and neural networks), and can adapt to different data distributions and task types (such as imbalanced data).

Common Use Cases

- Medical diagnosis: the integrated model integrates multi-source medical data (images, genes, clinical indicators) to improve the accuracy of disease detection, such as the early screening system for breast cancer.
- Financial risk control: In credit scoring and fraud detection, Boosting algorithms (such as LightGBM) enhance decision interpretability through feature importance analysis.
- Industrial predictive maintenance: Combining time series models with random forests to predict equipment failure cycles and reduce unplanned downtime losses.
- Natural language processing: Stacking integrates different text representation models such as BERT and TF-IDF to optimize sentiment analysis or machine translation performance.



02 Key Ensemble Learning Techniques



Bagging (Bootstrap Aggregating)

- Parallel training of weak learners: Bagging generates multiple sub datasets from the original dataset using bootstrap sampling. Each sub dataset is used to independently train a base learner (such as a decision tree), and the results are integrated through voting or averaging, significantly reducing model variance.

- Reduce the risk of over fitting: because the training data of the basic learner is obtained through random sampling, and the model results are output through aggregation, Bagging can effectively reduce the sensitivity of a single model to noise data, especially for high variance models (such as decision trees that are not pruned).

- Typical algorithm: Random forest is an extension of Bagging, which not only randomly samples samples samples, but also randomly selects features, further increasing the diversity of the base learner, improving the model's generalization ability and anti-interference ability.

Boosting (Adaptive Methods)

- Serial iterative optimization: Boosting trains a series of weak learners sequentially, and each new model adjusts its weight based on the erroneous samples of the previous model (such as AdaBoost's weighted error mechanism), gradually reducing the overall bias, and finally outputting a strong learner through weighted voting.

- Adaptive error correction: The algorithm will focus on misclassified samples and increase their weights in subsequent iterations, forcing the model to learn difficult to predict sample features, thereby improving overall accuracy (such as Gradient Boosting's gradient descent optimization).

- Sensitive to noise: Boosting amplifies the impact of erroneous samples, and if there is noise or outliers in the data, it may lead to overfitting of the model. Therefore, cross validation or early stopping strategies need to be used.

- Typical algorithm: XGBoost/LightGBM: an optimization algorithm based on gradient boosting framework, introducing regularization, feature bucket and other techniques to significantly improve training speed and prediction accuracy, widely used in Kaggle competitions and industrial scenarios.

Stacking (Meta-Learning)

Multi layer model integration

Stacking trains multiple base learners (first layer) and inputs their outputs as new features into meta learners (second layer, such as logistic regression or neural networks), using meta models to learn the combination strategy of base models and further improve prediction performance.

To avoid data leakage

To prevent the base model from "peeking" at the validation data during the training phase, K-fold cross validation is usually used to generate the output features of the base model, ensuring that the input data of the meta model is independent of the training set.

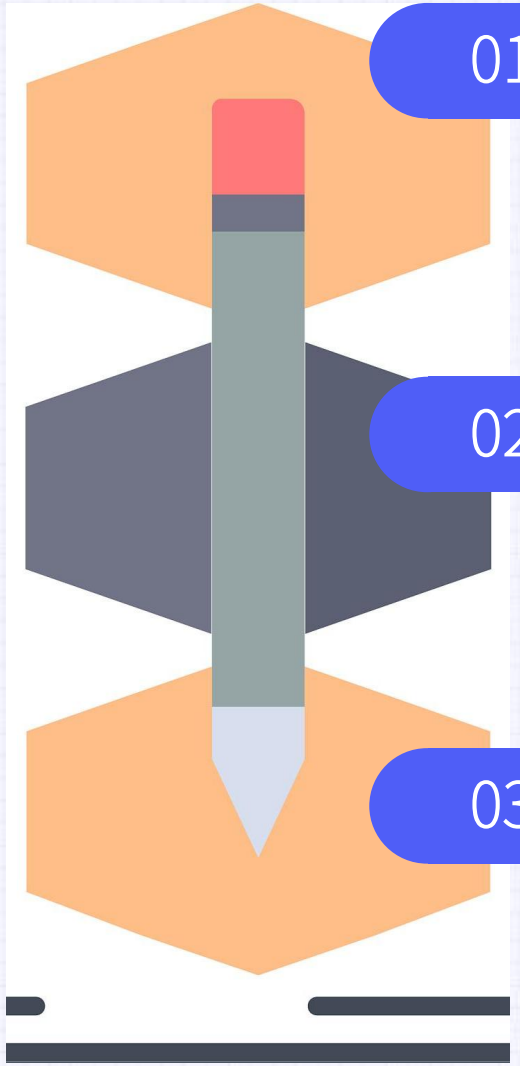
High computational complexity

Due to the need to train multiple sets of base models and meta models, Stacking has a high computational overhead and is typically used in scenarios that require extremely high prediction accuracy and sufficient resources, such as financial risk control and medical diagnosis.

03 Bagging and Random Forests



Bootstrap Sampling Process



01

Random sampling with replacement

Multiple subsets (Bootstrap samples) are generated from the original training set through random sampling with replacement. Each subset has the same size as the original dataset, but about 63.2% of the original samples are selected, while the rest are duplicate samples.

02

Sample independence

Each Bootstrap sample set is independently generated to ensure diversity in the training of the base learner, reducing the correlation (ρ) between models through differentiated data distribution.

03

Mathematical foundation

The probability of not being sampled is $1/e \approx 36.8\%$. Using the law of large numbers to ensure sampling stability provides theoretical support for reducing Bagging variance.

Random Forest Algorithm

Double randomness

Based on Bootstrap sampling, further random selection of features is performed (usually taking $\sqrt{p}$ or $\log_2 p$ features) to reduce the correlation between trees and enhance the robustness of the model.

Complete growth decision tree

Each base decision tree does not prune, allowing it to fully grow to pure leaf nodes, utilizing high variance characteristics to reduce overall variance through aggregation.

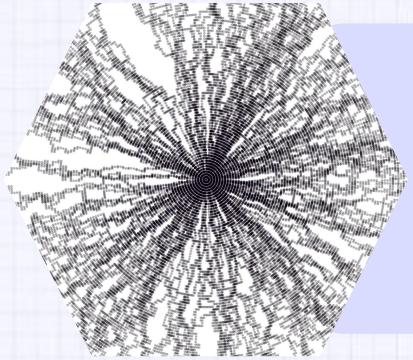
Voting/averaging mechanism

The classification task adopts the majority voting method (random selection in case of tie), and the regression task adopts mean prediction, combined with swarm intelligence to improve generalization ability.

Parallel training

Each tree is trained independently, supports distributed computing, significantly improves efficiency on large-scale data, and is suitable for high-dimensional feature scenarios.

Advantages Over Single Decision Trees

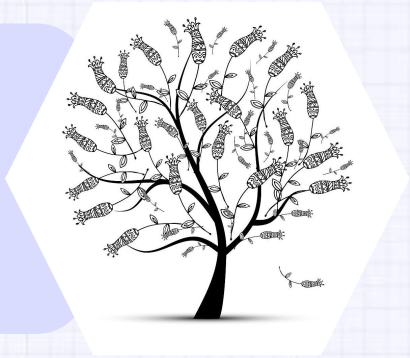


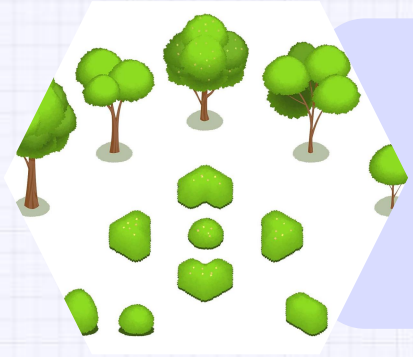
Reduce the risk of overfitting

By aggregating predictions from multiple trees, the noise sensitivity of a single tree can be offset, especially for high variance models (such as deep trees).

Processing high-dimensional data

feature random selection reduces the impact of dimension curse, automatically filters important features, and improves fault tolerance for redundant or irrelevant features.





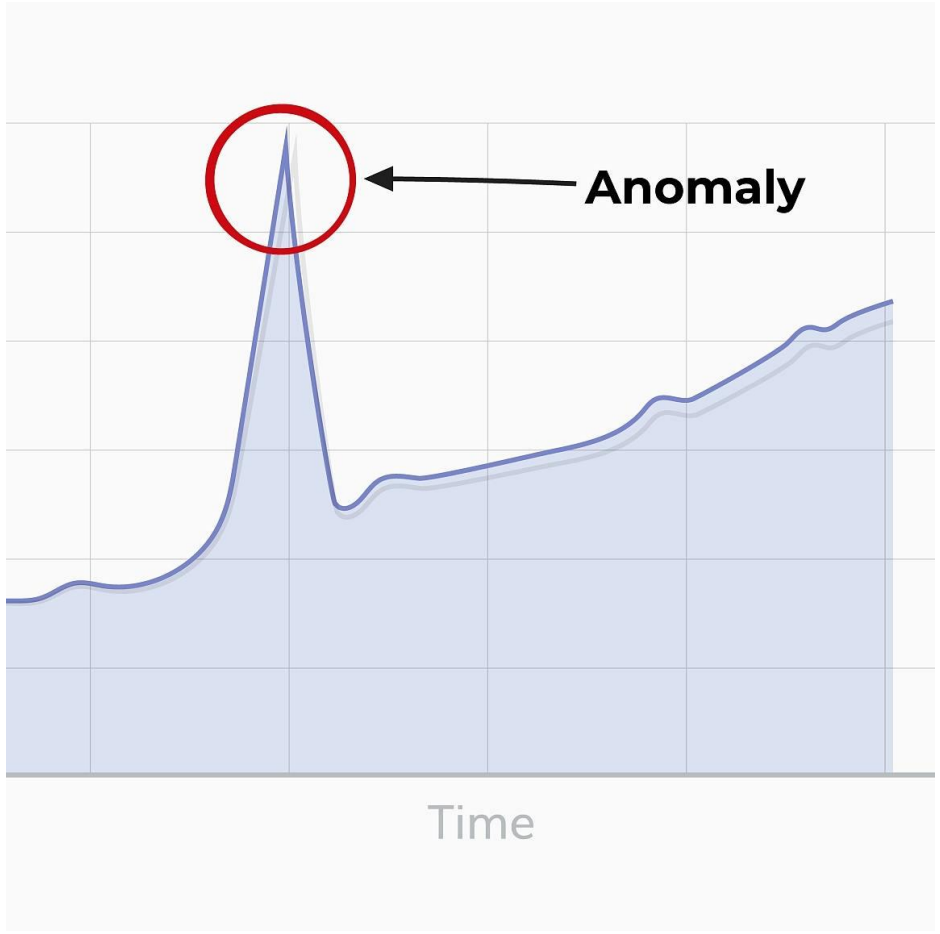
Natural interpretability

Provides feature importance assessment (such as Gini impurity reduction), which may not be as intuitive as a single tree, but is still superior to black box models (such as neural networks).

04 Boosting Algorithms



AdaBoost (Adaptive Boosting)



Iterative weighting mechanism

AdaBoost iteratively adjusts the weights of samples and classifiers, increasing the weight of misclassified samples and decreasing the weight of correctly classified samples in each round, so that subsequent classifiers focus more on difficult to classify samples. Finally, a strong classifier is formed by combining weak classifiers through weighted voting.

AdaBoost (Adaptive Boosting)

Adaptive error rate

The algorithm dynamically adjusts the contribution weight of each base classifier based on its weighted error rate. The lower the error rate, the higher the weight. This adaptability makes it highly robust to noisy data and outliers.

Exponential loss function

The exponential loss function is used as the optimization objective (L (y, F (x))=exp (- yF (x))), and the forward stepwise algorithm is used to minimize the loss. Its characteristic is that the penalty for misclassified samples increases exponentially, thereby accelerating the convergence of the model.

Gradient Boosting Machines (GBM)

- Gradient descent framework: GBM optimizes any differentiable loss function (such as mean square error, logarithmic loss, etc.) through gradient descent, fits the negative gradient (pseudo residual) of the current model in each iteration, gradually reduces the value of the loss function, and achieves model performance improvement.

- Decision Tree Based Learner: By default, CART regression trees are used as the base model. The complexity is controlled by limiting the tree depth (usually 3-6 layers), and each tree only learns a portion of the residuals (controlled by the learning rate η). This "incremental" training effectively prevents overfitting.

- Feature importance assessment: Based on the number of times features are used for splitting in all trees and the resulting purity improvement, the feature importance can be quantified, providing intuitive basis for feature selection, especially suitable for high-dimensional data analysis and scenarios with high interpretability requirements.





Gradient Boosting Machines (GBM)

Regularization technique

Introducing dual regularization of subsampling (row sampling/column sampling) and shrinkage, randomly selecting some samples or features for single round training, significantly improving the model's generalization ability, and becoming the basis for later improved algorithms such as XGBoost.

XGBoost and LightGBM

- Engineering optimization innovation: XGBoost optimizes the efficiency of splitting point search through pre sorted algorithm, supports parallel computing and sparse sensing, introduces second-order Taylor expansion approximation loss function, and adds L1/L2 regularization term, comprehensively surpassing traditional GBM in accuracy and speed.

- Histogram Acceleration Strategy: LightGBM adopts a histogram based decision tree algorithm to discretize continuous features into bins, significantly reducing memory usage and computational overhead. At the same time, it further improves training speed through single-sided gradient sampling (GOSS) and mutually exclusive feature bundling (EFB).

- Distributed computing support: Both support multi machine parallel training, XGBoost uses the Rabit framework to implement AllReduce communication, and LightGBM reduces communication overhead through voting parallel, enabling it to process TB level data and becoming the preferred solution for industrial scale large-scale applications.

05 Ensemble Learning Strategies



Voting (Hard & Soft)

- Hard Voting: Integrating the classification results of multiple base learners through a majority voting mechanism, where each base learner independently votes and ultimately selects the category with the highest number of votes as the ensemble output. Suitable for scenarios where the prediction results of base learners differ significantly and have similar performance.

- Soft Voting: A probability based weighted voting method that aggregates the average predicted probabilities of each base learner for each category and selects the category with the highest probability. Compared to hard voting, soft voting can reflect the uncertainty of the model in more detail, especially suitable for models with good probability calibration (such as logistic regression and random forest).

- Comparison of applicable scenarios: Hard voting is suitable for scenarios where the base learner outputs clear category labels (such as SVM), while soft voting is more suitable for models that output probabilities (such as neural networks) and usually achieves higher robustness.

Weighted Averaging

01

Weight allocation principle

dynamically allocate weights based on the performance differences of base learners (such as cross validation accuracy), and high-performance models obtain higher weights.

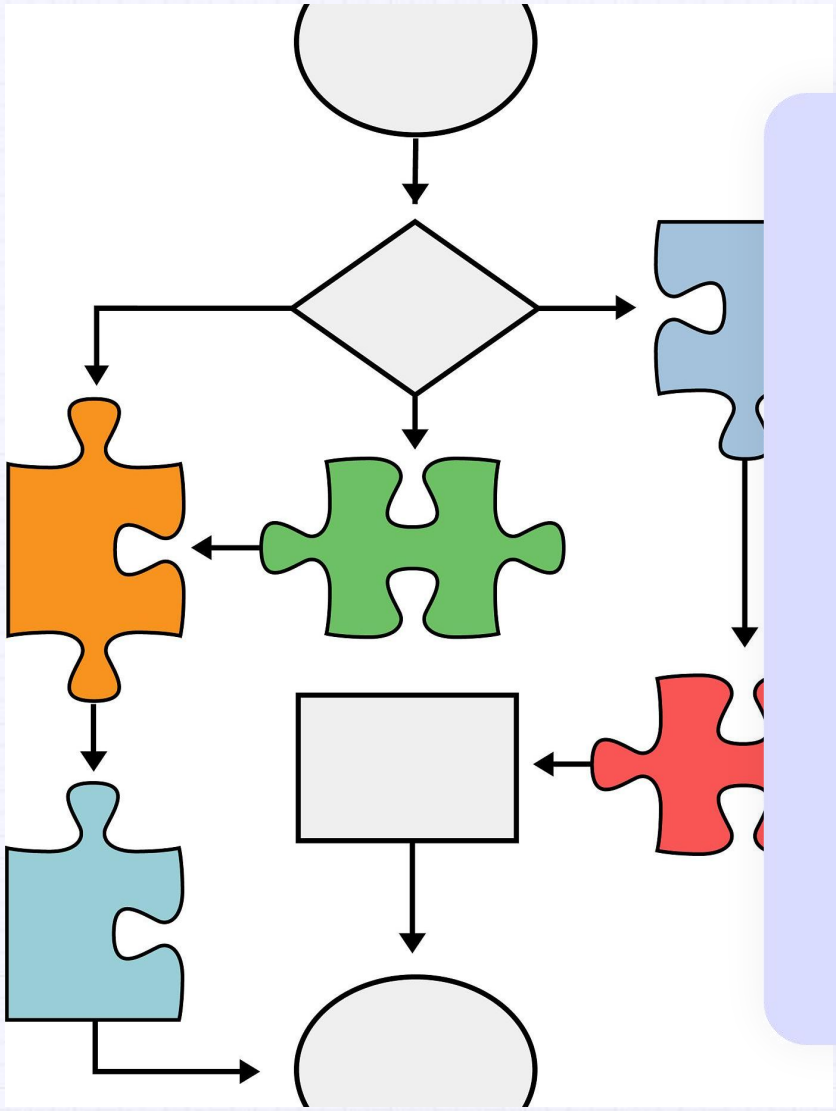
02

Dynamic weight optimization

The weights can be optimized through grid search or meta learning algorithms (such as gradient descent) to minimize the loss function of the ensemble model on the validation set. For example, Adaboost achieves dynamic weighting by iteratively adjusting the weights of erroneous samples.

The mathematical representation is a weighted sum: $y_{\text{ensemble}} = \sum_{i=1}^{M} w_i y_i$, where the weights w_i must satisfy $\sum_{i=1}^{M} w_i = 1$.

Weighted Averaging



Heterogeneous model integration

suitable for combining base learners with different structures (such as decision trees and linear models), attention should be paid to weight normalization to avoid scale differences.

Anti overfitting design

Regularization constraint weights (such as L2 regularization) can be used for noisy data or high variance models to prevent a few models from dominating the ensemble results.

Diversity Enhancement Methods

Input perturbation method

By using self sampling (Bagging) or feature subset sampling (such as column sampling in random forests), the base learner is forced to focus on different aspects of the data, improving the inter model variability.

Output perturbation method

For example, the Boosting series algorithm iteratively adjusts the sample distribution to focus subsequent models on the prediction errors of the preceding model, thereby generating highly complementary base learners.

Structural perturbation method

In neural network ensembles, diversity can be introduced through random initialization, Dropout, or different architecture designs (such as a mixture of ResNet and DenseNet).

06 Practical Applications



Kaggle Competition Case Studies

- Image classification task: Ensemble learning methods such as model fusion and stacking are widely used in Kaggle's CIFAR-10 image classification competition. The contestants significantly improved the classification accuracy by combining the prediction results of multiple convolutional neural network (CNN) models, and the final winning solution usually includes an ensemble of models such as ResNet and EfficientNet.
- Natural language processing challenges: In text emotion analysis or machine translation tasks, integrated learning technologies (such as model fusion of BERT, RoBERTa and XLNet) can effectively capture the semantic advantages of different models, reduce the deviation of a single model, and achieve higher scores in the ranking.
- Time series forecasting: In Kaggle's M5 Forecasting competition, integrated methods (such as the combination of LightGBM, XGBoost and Prophet) significantly improved the accuracy of commodity demand forecasting through weighted average or stacking strategies, and solved the problem of insufficient capture of seasonal fluctuations by a single model.

Financial Market Prediction

- Stock price prediction: Ensemble learning models (such as a mixture of random forests and LSTM) can combine the stability of traditional statistical methods with the temporal modeling ability of deep learning to provide more robust prediction results in volatile markets, helping to optimize quantitative trading strategies.
- Credit risk assessment: Banks and financial institutions use Boosting algorithms (such as XGBoost and CatBoost) to integrate multiple weak classifiers, accurately identify high-risk loan applications through feature importance and anomaly detection, and reduce default rates.
- High frequency trading signal generation: Integrated methods such as Bagging and Stacking are used to fuse multiple trading signal models, reduce noise interference, improve signal signal-to-noise ratio, and achieve higher returns in millisecond level trading.
- Cryptocurrency volatility analysis: By integrating ARIMA, Prophet, and neural network models, trading platforms can more accurately predict the price fluctuations of cryptocurrencies such as Bitcoin, assisting investors in formulating hedging strategies.

Medical Diagnosis Systems



Early cancer screening

integrated learning (such as the combination of random forest and support vector machine) performs well in the image diagnosis of breast cancer and lung cancer, reduces the misdiagnosis rate and improves the sensitivity of early disease detection through multi model voting mechanism.



Risk prediction of chronic disease

medical institutions use Stacking method to integrate electronic medical record data, genomics and lifestyle characteristics, build prediction models of diabetes or cardiovascular disease, and achieve personalized health intervention.

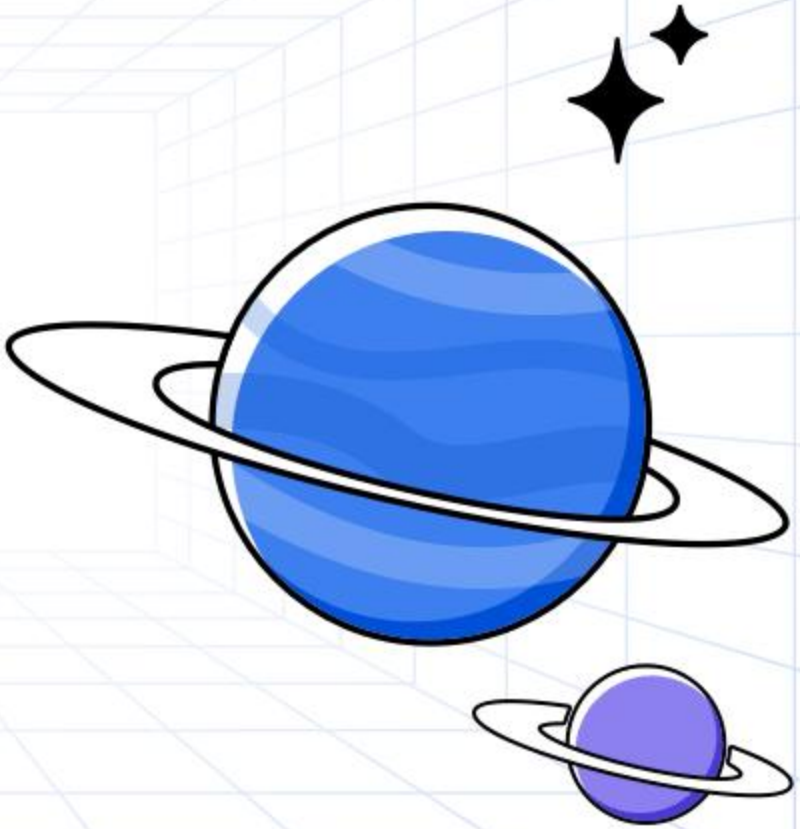


Epidemic transmission modeling

In COVID-19 prediction, integrating SEIR models with machine learning algorithms such as gradient boosting trees can more accurately simulate virus transmission trends and provide dynamic support for public health decision-making.

01

Introduction to Decision Trees



Definition and Basic Concepts

Tree structure model

Decision tree is a supervised learning algorithm based on a tree structure, consisting of root nodes, internal nodes, and leaf nodes. It achieves classification or regression tasks by recursively segmenting data.

Splitting rule

Each internal node represents a judgment condition for a feature attribute, and the optimal splitting point is selected based on indicators such as information gain and Gini impurity to divide the dataset into purer subsets.

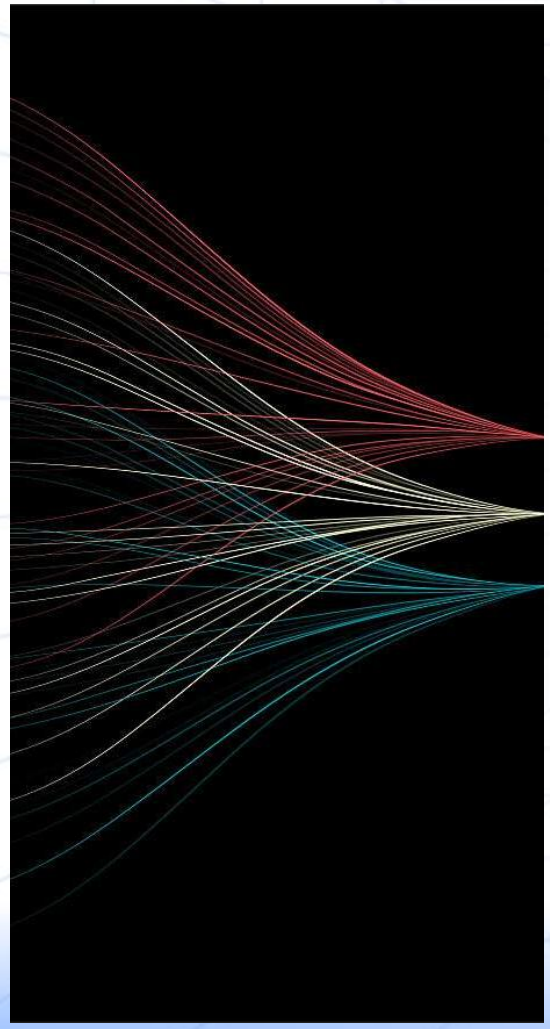
Termination condition

Stop splitting when node samples belong to the same category, reach the preset depth, or have too few samples, generate leaf nodes, and output prediction results.

Interpretability

The logic of decision trees is intuitive and clear, and the prediction process can be explained through path tracing. It is suitable for scenarios that require transparent models, such as medical diagnosis.

Applications in Machine Learning



Classification task

Widely used in scenarios such as spam email recognition and customer churn prediction, quickly classify sample categories through feature judgment.

Regression analysis

such as predicting housing prices, using the mean or median of continuous values as leaf node outputs to **process non-linear relationship data**.

Feature importance assessment

By calculating the contribution of features in splitting, it assists in data dimensionality reduction or business decision-making (such as **key variable analysis in financial risk control**).

Advantages and Limitations

High training efficiency

Strong adaptability to large-scale data, low preprocessing requirements (no need for normalization), and support for parallelization optimization (such as random forest).

Overfitting risk

If the tree depth is too large or there is a lot of data noise, it may capture the details of the training set and lead to a decrease in generalization ability, which needs to be alleviated by pruning or setting hyperparameters.

Category imbalance sensitivity

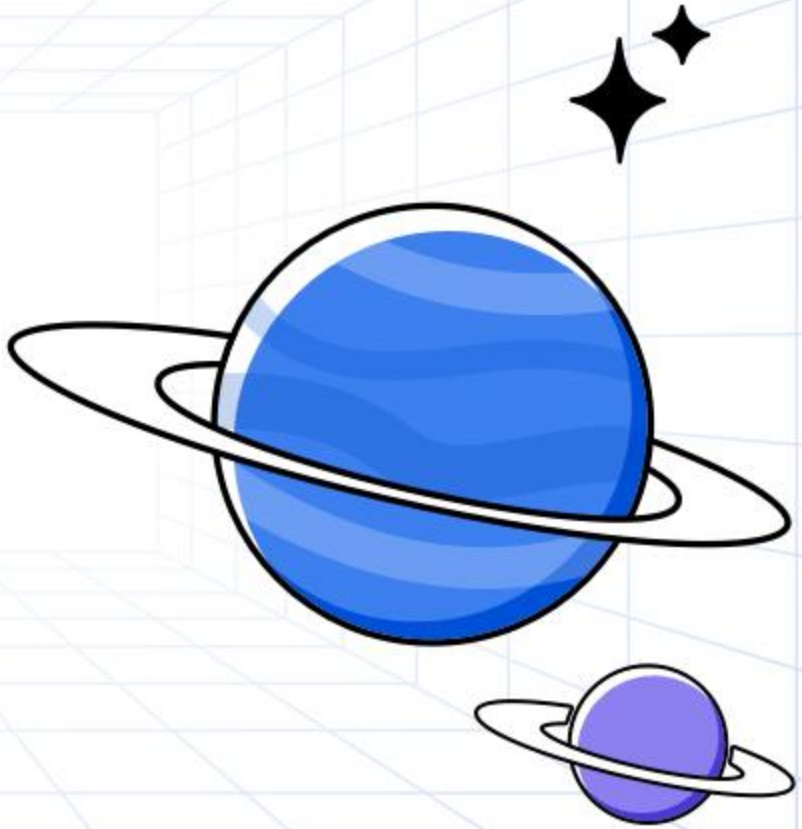
Minority class samples may be ignored during splitting, and weighted loss functions or sampling techniques need to be used to balance the data distribution.

Boundary constraint

The decision boundary is axis parallel (rectangular partition), which makes it difficult to fit oblique or circular complex patterns. Therefore, ensemble methods need to be combined to improve performance.

02

Key Components of Decision Trees



Decision Nodes

Logical judgment core

Decision nodes represent key judgment points in the model, and each node performs binary or multivariate partitioning of data based on specific features (such as age ⩾ 30 or income level).

For example, in medical diagnosis, the decision node may preliminarily screen fever patients based on the condition of **"body temperature>38 ° C**.

Split criteria optimization

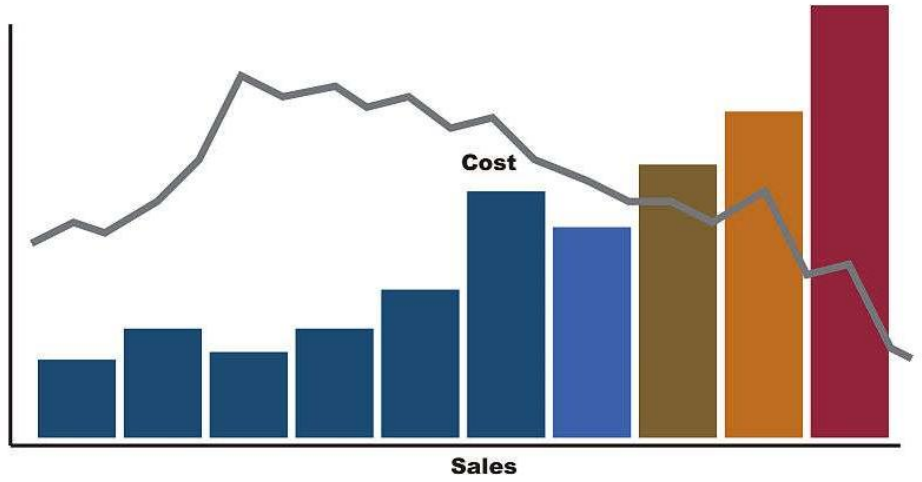
Node splitting relies on indicators such as **information gain** and **Gini impurity**, and modern algorithms (such as C4.5) **dynamically select the optimal features**. In practical applications, overfitting issues need to be considered, often by **limiting the maximum depth or minimum sample size** through **pre pruning**.

Chance Nodes

Probabilistic path selection

In **Bayesian decision trees**, opportunity nodes introduce probability branches (such as **"treatment plan A has a 60% success rate"**), which need to be combined with prior distributions to calculate expected utility.

This type of node is commonly used in **financial risk control** models to evaluate the default **probability of different repayment strategies**.



Chance Nodes

Uncertainty handling

For missing data, opportunity nodes can use **multiple imputation** or **surrogate splits techniques**. For example, in retail forecasting, when customer income data is missing, **alternative features such as shopping frequency are used for inference**.

Dynamic weight adjustment

The decision tree driven by **reinforcement learning** will update the node probability weight in real time, for example, the auto drive system will adjust the confidence threshold of steering decision according to the real-time road conditions.

Terminal Nodes (Leaves)



Classification/Regression Output

Leaf nodes store the final prediction results, **classification tasks** output category labels and probabilities (such as "malignant tumors: 92% confidence"), and **regression tasks** output continuous values (such as "house price estimate=¥ 4.2M ± 5%").

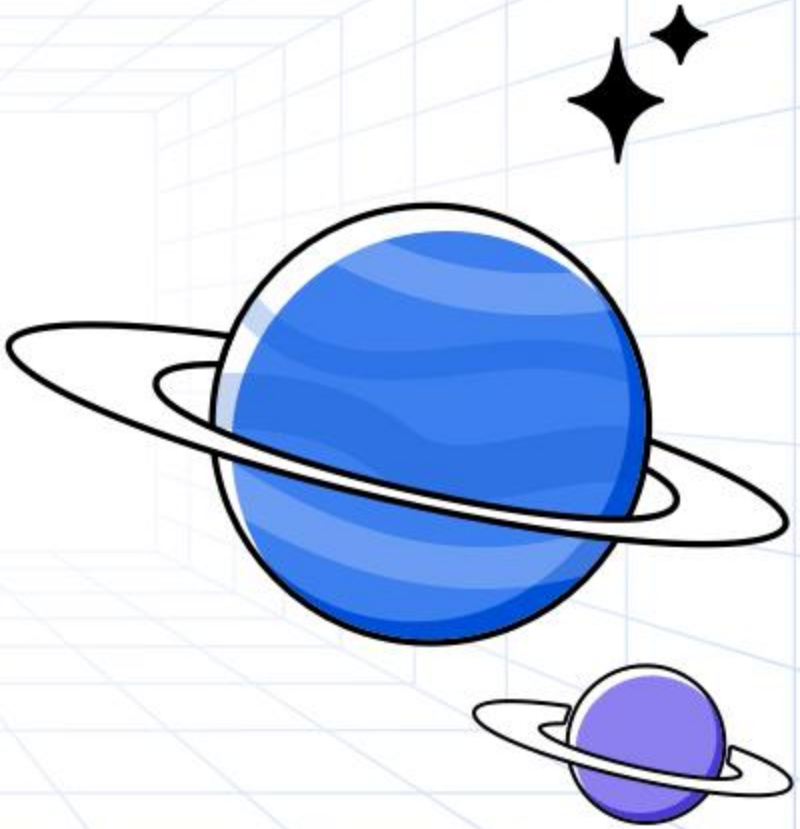


Business rule mapping

In explainable AI scenarios, leaf nodes often correspond to **specific business actions**, such as **"reject loan" or "approve 500000 limit" in credit approval**, which require collaboration with the compliance department to define thresholds.

03

Decision Tree Algorithms



ID3 (Iterative Dichotomiser 3)

- **Feature selection based on information gain:** The ID3 algorithm selects the optimal splitting node by calculating the information gain (Information Gain) of each feature, which represents the degree of reduction in data uncertainty after splitting. The feature with the highest gain is selected as the partitioning criterion for the current node.

- **Only supports discrete features:** The ID3 algorithm requires that the input features must be discrete values and cannot directly handle continuous features or missing values. Preprocessing (such as binning or padding) is required to solve this problem.

- **Easy overfitting:** Due to ID3's tendency to select features with multiple values (with information gain biased towards high cardinality features), the generated tree may be too complex and requires pruning (such as pre pruning or post pruning) to optimize generalization ability.

The ID3 algorithm uses entropy and information gain to decide how to split the data at each node.

How ID3 Uses These

1. Compute entropy of the current dataset.
2. For each unused feature, compute information gain.
3. Choose the feature with the highest information gain to split the data.
4. Repeat recursively on each subset until:
    - All samples in a node belong to the same class, or
    - No more features are left to split.

🔢 1. Entropy (Impurity Measure)

For a dataset S with c classes:

$$\text{Entropy}(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Where:

- p_i is the **proportion** of samples belonging to class i in S.

📈 2. Information Gain

For a feature A, the **information gain** is:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

Where:

- $\text{Values}(A)$: all possible values of feature A
- S_v: subset of S where feature A has value v
- $|S_v|$: number of samples in S_v
- $|S|$: total number of samples

Example: Watermelon Classification (Simplified Dataset)

Dataset

Sample	Texture (Feature)	Root (Feature)	Good Melon? (Class)
1	Clear	Curled	Yes
2	Clear	Curled	Yes
3	Blurred	Stiff	No
4	Blurred	Curled	No

Step-by-Step Execution

Step 1: Root Node (All 4 samples)

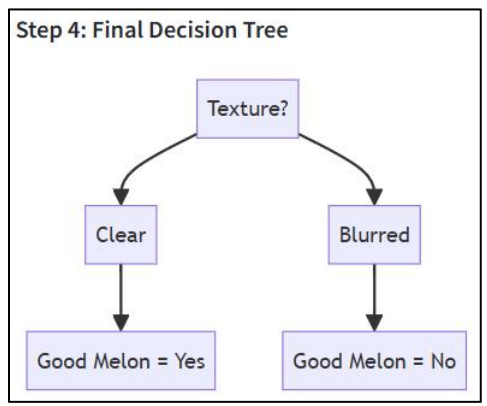
- Class distribution: 2 "Yes", 2 "No"
- Entropy: $H(S) = -\left(\frac{2}{4}\log_2\frac{2}{4} + \frac{2}{4}\log_2\frac{2}{4}\right) = 1.$

Step 2: Calculate Information Gain for Features

- **Texture feature**:
  - "Clear" (Samples 1,2): Both "Yes" → $H(S_{\text{Clear}}) = 0$
  - "Blurred" (Samples 3,4): Both "No" → $H(S_{\text{Blurred}}) = 0$
  - Gain: $\text{Gain}(\text{Texture}) = 1 - \left(\frac{2}{4}\times 0 + \frac{2}{4}\times 0\right) = 1$
- **Root feature**:
  - "Curled" (Samples 1,2,4): Classes = [Yes, Yes, No] → $H(S_{\text{Curled}}) = -\left(\frac{2}{3}\log_2\frac{2}{3} + \frac{1}{3}\log_2\frac{1}{3}\right) \approx 0.918$
  - "Stiff" (Sample 3): "No" → $H(S_{\text{Stiff}}) = 0$
  - Gain: $\text{Gain}(\text{Root}) = 1 - \left(\frac{3}{4}\times 0.918 + \frac{1}{4}\times 0\right) \approx 0.311$
- **Result**: Texture has higher gain → **split on Texture**.

Step 3: Split Root Node

- **Child Node 1 (Texture = Clear)**:
  - Samples 1,2 → both "Yes" → **leaf node "Yes"**.
- **Child Node 2 (Texture = Blurred)**:
  - Samples 3,4 → both "No" → **leaf node "No"**.



Keynote for ID3

1.Why ID3?

- Uses information gain to select features that maximize reduction in uncertainty.
- Simple and interpretable for discrete features.

2.Limitations

- Bias toward multi-valued features: Features with many values may artificially inflate gain.
- No continuous features: Requires discretization first.
- No pruning: Risk of overfitting (addressed in C4.5).

C4.5 and C5.0

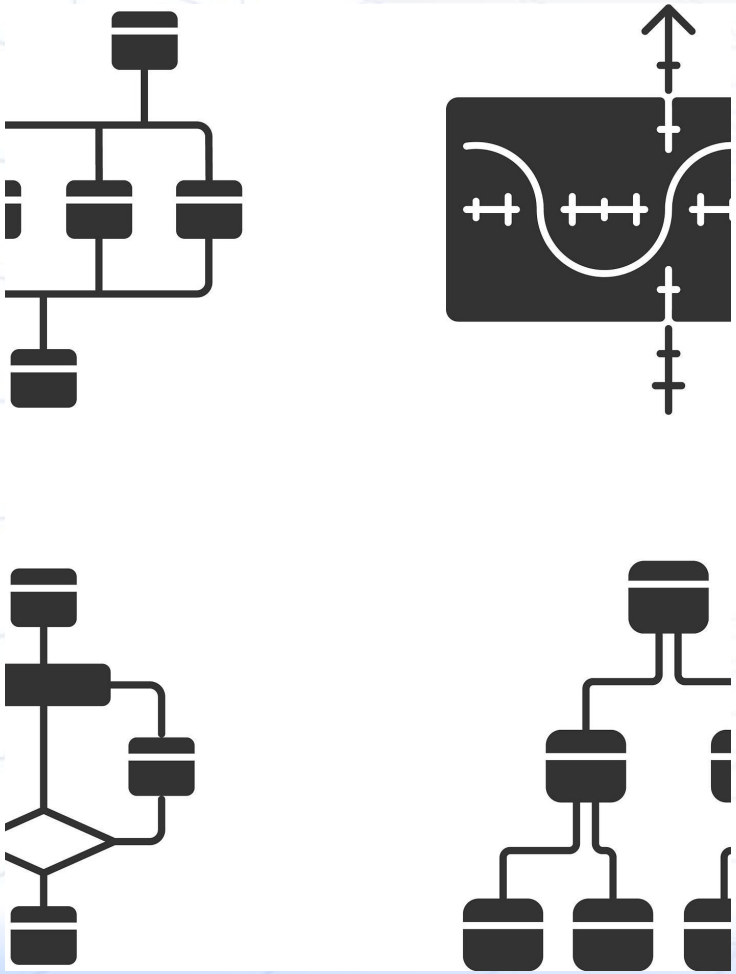
Improved information gain ratio

C4.5 introduces **information gain ratio** instead of information gain, which corrects the preference for **high cardinality features** by considering the entropy of the features themselves (splitting information), and improves the balance of splitting.

Support for continuous features and missing values

C4.5 can automatically **discretize continuous features into threshold partitions** and **support missing value processing (such as assigning branches based on probability)**, expanding the applicability of the algorithm.

C4.5 and C5.0



Pruning optimization

C4.5 adopts **pessimistic pruning**, which **removes unreliable branches** based on statistical significance testing and significantly reduces the risk of overfitting.

Enhancement of C5.0

As a commercial version, C5.0 further optimizes computational efficiency (such as **boosting integration**) and **memory management**, and supports more flexible rule generation and weight adjustment.

1. Entropy of a Set

For a dataset S with classes $i = 1, 2, ..., k$, let p_i be the proportion of instances in class i.

Then the **entropy** of S is:

$$\text{Entropy}(S) = -\sum_{i=1}^{k} p_i \log_2 p_i$$

---

2. Information Gain

Let A be an attribute with values $v \in \text{Values}(A)$.

Let S_v be the subset of S where attribute A has value v.

Then the **information gain** from splitting S on attribute A is:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

3. Gain Ratio (C4.5 Split Criterion)

To normalize the gain and avoid bias toward attributes with many values, C4.5 uses **gain ratio**:

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}(S, A)}$$

Where **split information** is:

$$\text{SplitInfo}(S, A) = -\sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$$

Key Points

- Entropy, Gain, and GainRatio are computed exactly as defined earlier
- Numeric attributes are tried at every observed value midpoint; only the best cut is kept
- Missing values (if present) are handled probabilistically during both training and classification, but are omitted from the pseudocode for clarity

This recursive procedure builds the full C4.5 tree; post-pruning may follow to reduce overfitting.

C4.5 Main Procedure (English pseudocode)

Input

- **S**: set of labelled training instances
- **Attrs**: set of candidate attributes (may include numeric ones)

Output

- **Tree**: a decision tree that classifies every instance in S

Procedure C4.5(S, Attrs)

1. **Create a node N**
2. **Base-case checks**
   a. If every instance in S has the same class C → return N as a leaf labelled C
   b. If Attrs is empty → return N as a leaf labelled with the majority class in S
   c. If S is empty → return N as a leaf labelled with the majority class of the parent node
3. **Select the best split**
   a. For every attribute A ∈ Attrs
   - Compute GainRatio(S, A)
     - Discrete attribute: use original values
     - Numeric attribute: sort values; try every binary cut-point; keep the cut with the maximum gain ratio

   b. Let A* be the attribute with the highest gain ratio (must exceed 0)
   c. Label N with the test on A*
4. **Partition the data**
   a. Discrete A*:
   For each value v of A*
   - Sv = {instances in S where A* = v}
   - Childv = C4.5(Sv, Attrs \ {A*})
   - Attach Childv to N under branch v

   b. Numeric A* (cut-point t):
   - S≤t = {instances in S with A* ≤ t}
   - S>t = {instances in S with A* > t}
   - Attach C4.5(S≤t, Attrs) to N under branch "≤ t"
   - Attach C4.5(S>t, Attrs) to N under branch "> t"
5. **Return the completed subtree rooted at N**

Key C5.0 features: Automatic continuous feature handling, boosting support, rule generation

Gain Ratio (C5.0's Key Metric):

$$GainRatio(A) = \frac{IG(A)}{SplitInfo(A)}$$

C5.0 Enhancements Over C4.5

1.Automatic Continuous Feature Handling:

●Finds optimal splits for continuous features (e.g., **Humidity ≤ 75**) without manual intervention.

1.Boosting Support:

●Builds multiple trees with weighted voting (e.g., 10-tree ensemble) to improve accuracy.

1.Rule Generation:

Converts trees to simplified rules:

Rule 1: IF Outlook=Overcast THEN Play=Yes
Rule 2: IF Outlook=Sunny AND Humidity≤75 THEN Play=Yes
Rule 3: IF Outlook=Rainy AND Windy=False THEN Play=Yes
ELSE Play=No

1.Efficiency Optimizations:

●100x faster than C4.5 on large datasets.

●90% less memory usage.

CART (Classification and Regression Trees)

1 Gini coefficient and squared error

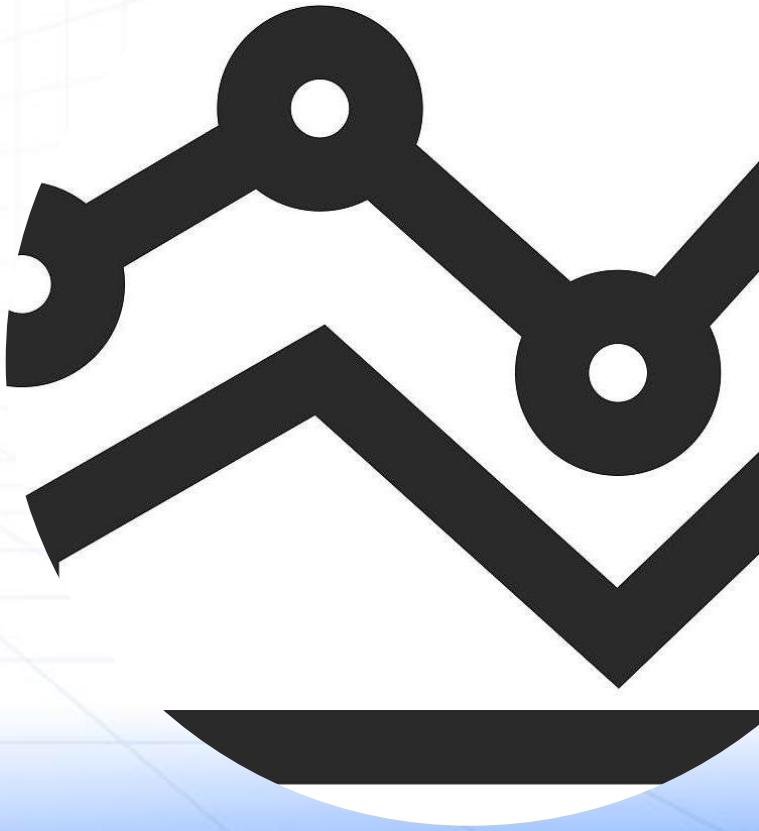
CART uses the **Gini index** to measure the impurity of classification tasks, while regression tasks use the **minimum squared error (MSE)** as the splitting criterion to achieve **a unified framework for classification and regression**.

2 Binary tree structure

CART **only generates two child nodes per split (binary partitioning)**, simplifying the tree structure through recursive binary partitioning to improve the interpretability and computational efficiency of the model.

3 Cost Complexity Pruning

CART uses **Cost Complexity Pruning** to select the optimal subtree through cross validation, balancing model complexity and prediction accuracy.



CART Classification Tree

1. Gini Impurity

For any node τ containing n samples with class proportions
$p_j = |\{i : y_i = j\}| / n, \quad j = 1,\dots,J$:

$\mathrm{Gini}(\tau) \triangleq 1 - \sum_j p_j^2$.

2. Split Score

Let a binary split s generate left node L and right node R.
Define the impurity decrease:

$\Delta\mathrm{Gini}(s) \triangleq \mathrm{Gini}(\tau) - (|L|/|\tau|)\cdot\mathrm{Gini}(L) - (|R|/|\tau|)\cdot\mathrm{Gini}(R)$.

The optimal split is
$s^* = \operatorname{argmax}_{s\in\mathcal{S}} \Delta\mathrm{Gini}(s)$,
where $\mathcal{S}$ is the set of all candidate binary splits on every attribute.

3. Cost-Complexity Pruning

For a subtree $T \subset T_{max}$ define
$R_\alpha(T) \triangleq R(T) + \alpha|T|, \quad \alpha \geq 0$,
with empirical mis-classification cost
$R(T) = \sum_{t\in\mathrm{leaves}(T)} (1 - \max_j p_j(t))\cdot n(t)$.

The minimal-cost subtree is
$T_\alpha = \operatorname{argmin}_{T\subseteq T_{max}} R_\alpha(T)$,
with α selected by cross-validation.

4. Prediction Rule

For a new instance x let $\ell(x)$ be the leaf reached.
Predicted class:
$\hat{y}(x) = \operatorname{argmax}_j p_j(\ell(x))$.

Recursive Construction for CART

```
CART(S, A):
    create node τ
    if ∃ j : yᵢ = j ∀ i ∈ S:
        return leaf(τ, j)
    if A = ∅ or |S| < n_min:
        return leaf(τ, mode{y : (x,y) ∈ S})

    (s*, Δ*) ← argmax_{s∈𝒮_A} ΔGini(s)
    if Δ* ≤ 0:
        return leaf(τ, mode{y : (x,y) ∈ S})

    (L, R) ← partition(S, s*)
    τ.test  ← s*
    τ.left  ← CART(L, A)
    τ.right ← CART(R, A)
    return τ
```

Prune T_max via minimal R_α to obtain the final CART model.

Gini Impurity Formula:

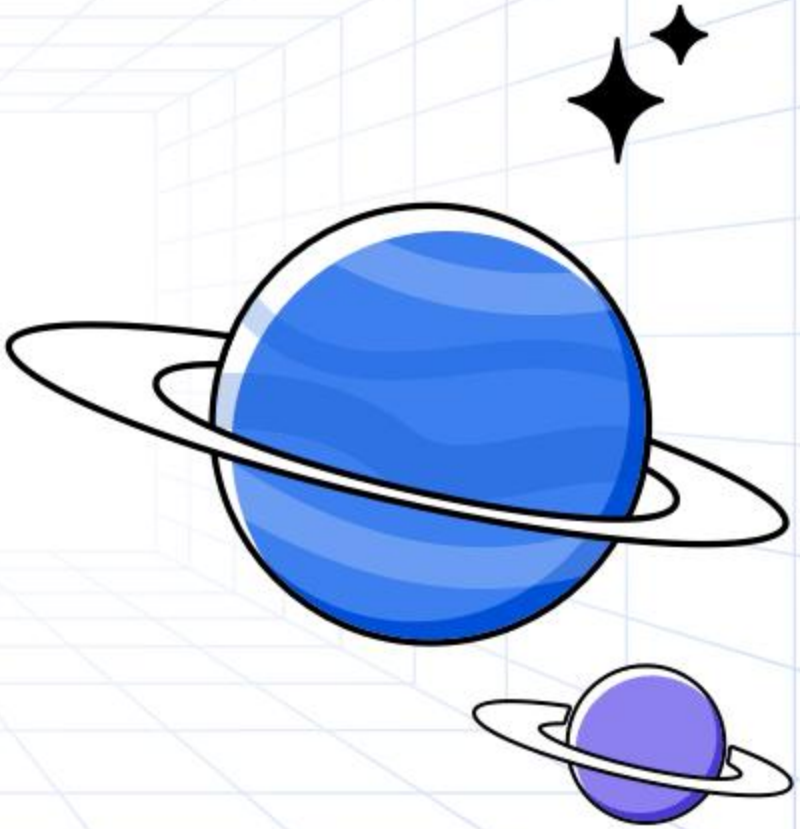
$$Gini(S) = 1 - \sum_{i=1}^{c} (p_i)^2$$

Where p_i = proportion of class i in subset S.

Overall Gini: $1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$

04

Decision Tree Generation Process



Node Splitting Criteria



Information Gain

Evaluating the splitting effect by calculating the entropy difference of the dataset before and after splitting, selecting features that can maximize information gain as splitting nodes, suitable for classification problems. The more entropy decreases, the better the splitting effect.



Gini Impurity

measures the degree of disorder in a dataset, with a lower Gini value indicating higher purity. Decision tree selection splits the feature that minimizes the total Gini impurity of child nodes, commonly used in CART algorithm.



Chi Square Test

Used for evaluating the splitting of classification features by selecting splitting nodes based on the independence of statistical features from the target variable. The larger the Chi Square value, the more significant the splitting.

Handling Continuous and Categorical Data

- **Continuous feature discretization:** Convert continuous values into discrete intervals using binary methods (such as C4.5 algorithm) or binning methods, such as selecting the median as the segmentation point and recursively partitioning until the stopping condition is met.

- **One Hot Encoding:** Binary vector transformation of multi class features to avoid algorithmic misjudgment of numerical relationships between classes, but may increase tree depth and computational complexity.

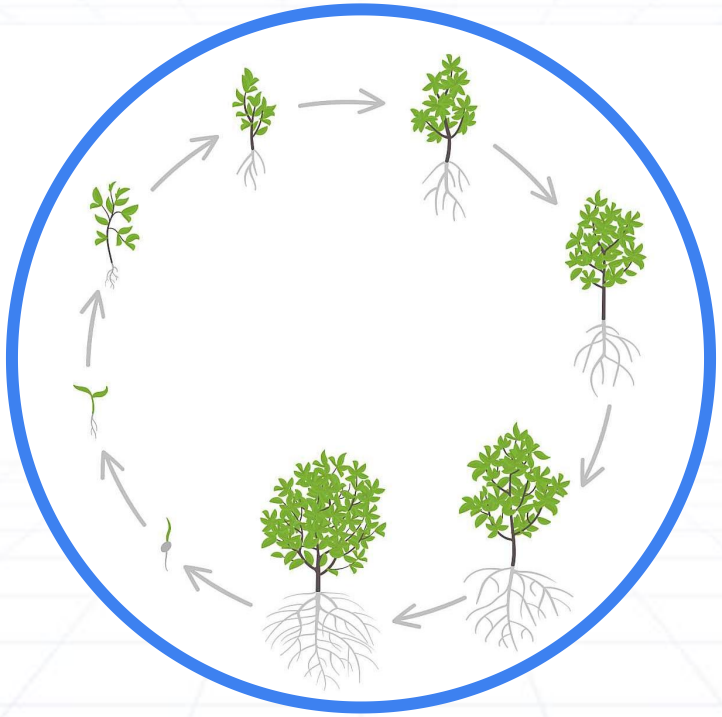
- **Target Encoding:** Using statistical measures of the target variable (such as mean) to replace category values, suitable for high cardinality classification features, but must prevent overfitting.

- **Variance Reduction:** In a regression tree, select the continuous feature segmentation points that minimize the variance of the target variable in the child nodes, ensuring the stability of the prediction results.

Stopping Conditions (Tree Pruning)

Max Depth Limit

Presets the maximum level of the tree to prevent overfitting of the model, but requires adjusting hyperparameters through cross validation to balance bias and variance.



Min Samples Split

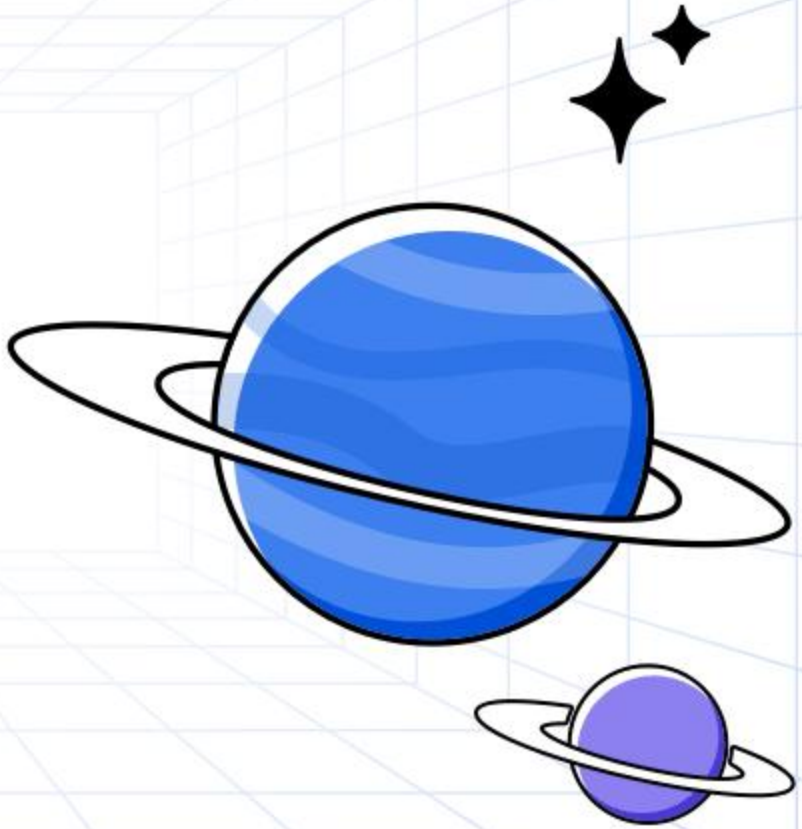
When the number of samples in a node is below a threshold, the split is terminated to avoid generating subtrees with no statistical significance. It is usually set to 5-20.

Post pruning

First, form a complete tree, and then merge redundant nodes from bottom to top (such as cost complexity pruning), using validation set accuracy as the pruning basis to improve generalization ability.

05

Practical Implementation Steps



Data Collection and Preparation

Data source selection

The quality of decision tree models is **highly dependent on data quality**, and reliable and diverse data sources should be selected, such as public datasets (such as UCI, Kaggle) or internal enterprise databases, to ensure that the data covers the key features of the target problem.

Feature engineering processing

data cleaning (processing missing values, outliers), encoding (such as single hot encoding categorical variables), and standardization (numerical features), while screening important features through correlation analysis or domain knowledge to reduce noise interference.

Dataset partitioning

Divide the data into **training set (70-80%), validation set (10-15%), and testing set (10-15%)** to ensure the independence of model training, tuning, and evaluation, and avoid overfitting issues.

Model Training and Evaluation

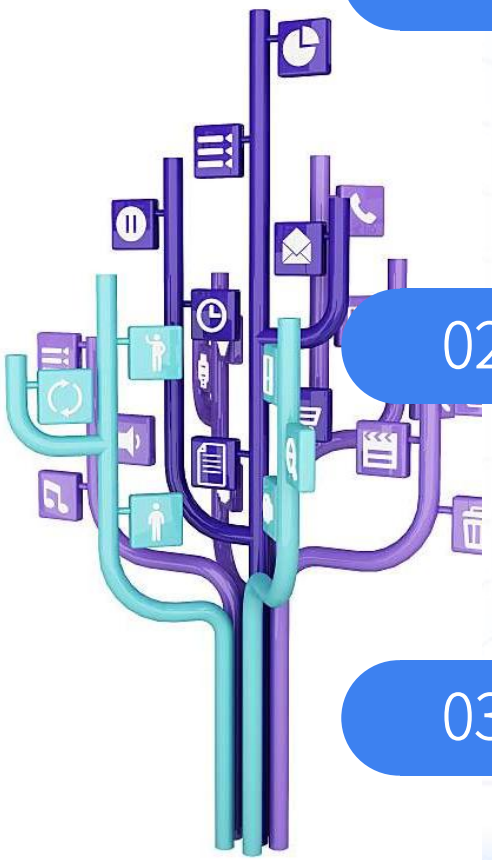
- **Algorithm selection and parameter setting:** Select a suitable decision tree algorithm (such as CART, ID3, C4.5) based on the **task type (classification or regression)**, and set **key parameters (such as maximum depth, minimum sample splitting number)** to balance model complexity and generalization ability.

- **Training process optimization:** Evaluate model stability through cross validation (such as K-fold cross validation), optimize hyperparameters through grid search or random search, and improve model performance (such as accuracy and F1 score).

- **Overfitting control:** using pruning techniques (pre pruning or post pruning) to limit the branch depth of the tree, or using ensemble methods (such as random forest) to reduce the variance of a single decision tree.

- **Evaluation indicator analysis:** using confusion matrix, accuracy, recall and other indicators for classification tasks; The regression task focuses on mean square error (MSE) and R^2 values to ensure the reliability of the model in practical scenarios.

Visualizing Decision Trees



01

Application of graphical tools

Using Python libraries such as' sklearn. tree. plot_tree 'or' Graphviz 'to generate tree structure diagrams, **visually displaying node splitting rules, feature importance, and sample distribution**, making it easy for non-technical personnel to understand.

02

Critical path interpretation

By **highlighting the decision path** (such as the path from the root node to a specific leaf node), analyze the logical basis of the model for specific prediction results, and enhance the interpretability of the model.

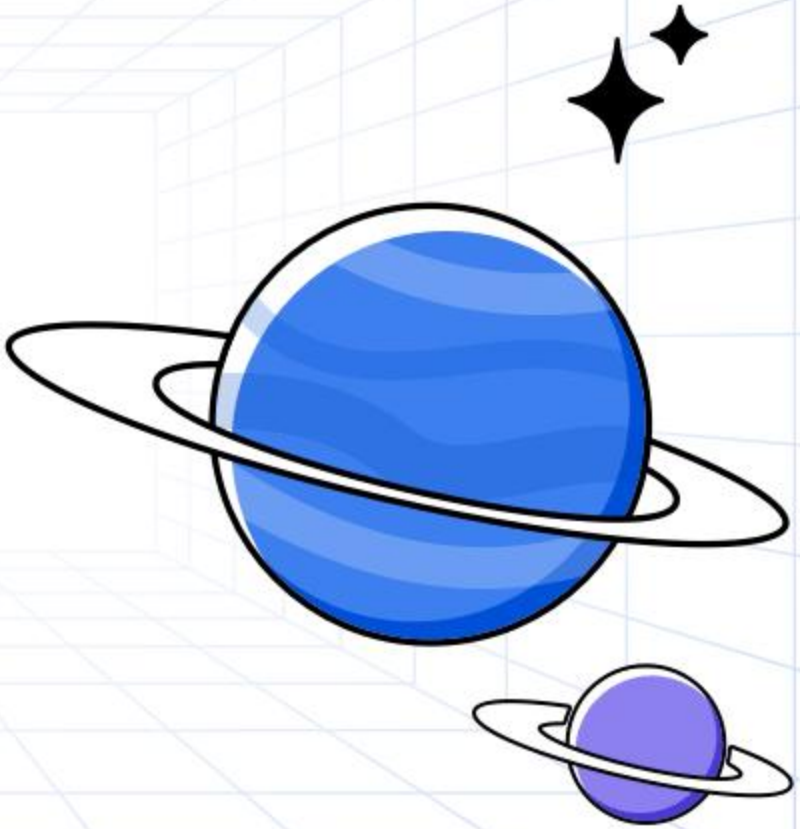
03

Feature importance ranking

Visualize feature importance scores (such as **Gini impurity reduction**) to help identify the features that have the greatest impact on prediction results and guide subsequent feature engineering optimization.

06

Applications and Case Studies



Classification Problems (e.g., Customer Churn)

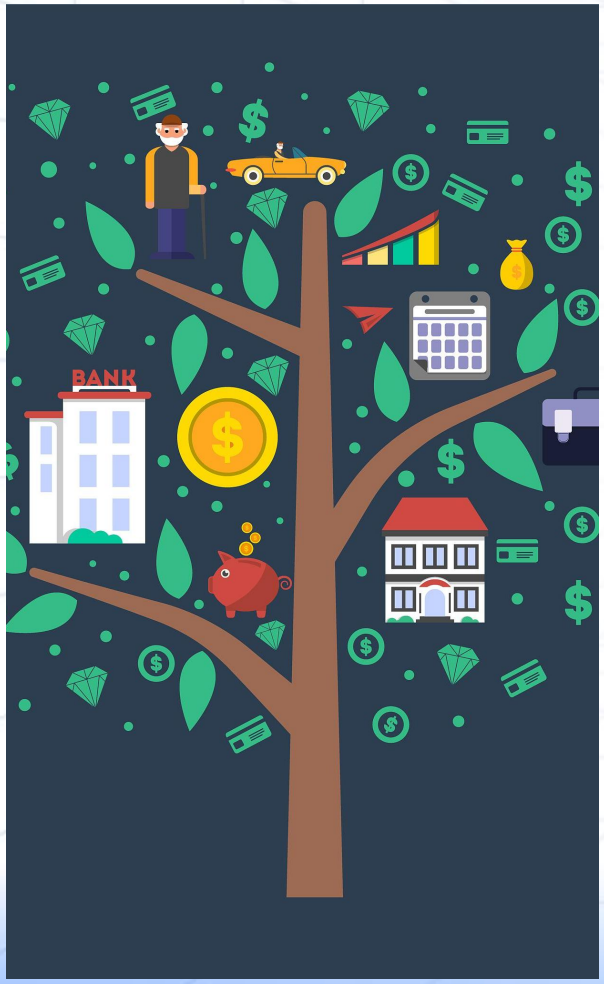
- **Customer churn prediction:** Decision trees analyze user behavior data (such as login frequency and consumption records), classify feature importance (such as the most recent consumption time), accurately identify high-risk churn customers, and help companies develop targeted retention strategies.

- **Credit risk assessment:** Financial institutions use decision trees to stratify borrowers' income, liabilities, credit history, and other characteristics, automatically classify high-risk and low-risk customers, and optimize loan approval processes.

- **Junk email filtering:** Based on the frequency of keywords and sender reputation in email content, decision trees can quickly classify junk emails from normal emails, improving filtering efficiency to over 90%.

- **Early screening of diseases:** in the medical field, the decision tree integrates symptom indicators (such as blood glucose value and blood pressure) and demographic data to assist doctors in rapid preliminary screening of patients with diabetes or hypertension.

Regression Problems (e.g., Housing Prices)



House price prediction model

Decision tree regression analysis of house characteristics (area, location, age), quantifying the impact of different factors on prices through splitting nodes, with an error rate controlled within 5% -8%.

Sales trend analysis

The retail industry combines historical sales data with promotional activities and seasonal factors to construct a decision tree regression model, predict future quarterly sales, and optimize inventory management.

Energy consumption estimation

Based on building type, weather data, and equipment usage time, decision tree regression can accurately predict the monthly electricity consumption of commercial buildings, helping with energy-saving and emission reduction planning.

Real-world Use Cases

- **Breast cancer diagnosis:** the decision tree integrates the tumor size, cell morphology and other pathological characteristics, assists doctors to distinguish benign/malignant tumors, with an accuracy rate of 85% -92%, significantly reducing the misdiagnosis rate.

- **Financial fraud detection:** By constructing decision trees based on transaction amount, location, frequency, and other characteristics, abnormal transactions (such as multi country consumption in a short period of time) are marked in real time to improve the response speed of the bank's anti fraud system.

- **Crop disease identification:** Agricultural expert systems use decision trees to analyze leaf images, soil moisture, and other data to quickly diagnose rice sheath blight or wheat rust, and recommend precise pesticide application plans.

- **Recommendation system optimization:** E-commerce platforms use decision trees to divide user interest tags (browsing history, purchase preferences), personalize product recommendations, and increase click through rates by 30% -50%.

01

Introduction to Decision Tree Regression

Definition and Purpose

Supervised learning method

Decision tree regression is a supervised learning algorithm used to predict continuous target variables by recursively dividing the dataset into smaller subsets to establish a tree model.

Non parametric models

Unlike linear regression, decision tree regression does not assume that data follows a specific distribution and can automatically capture nonlinear relationships and feature interactions.

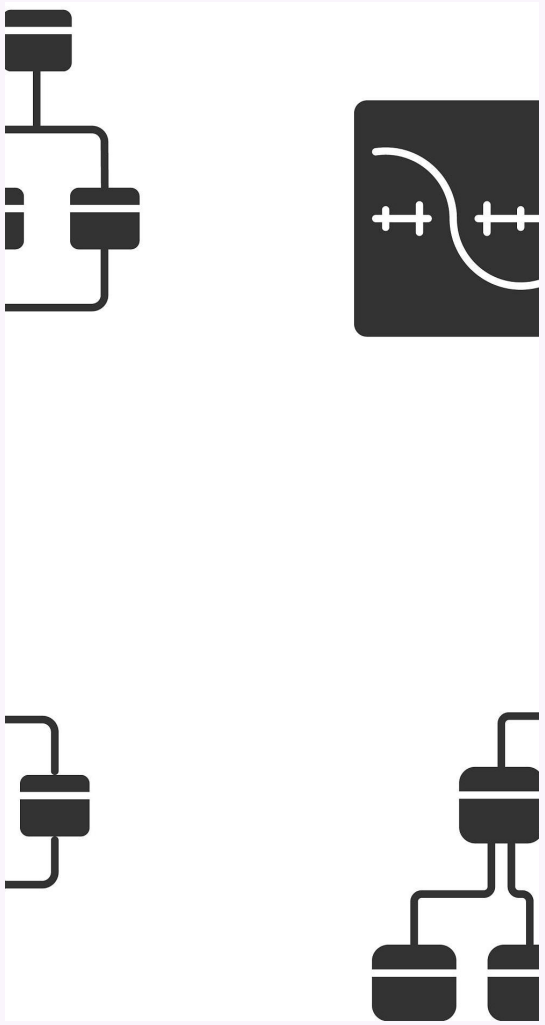
Strong interpretability

The generated tree structure is intuitive and can explain predictive logic through node splitting rules, making it suitable for business scenarios that require transparency, such as financial risk control.

Handling mixed features

Supports both numerical and categorical features, and selects the optimal splitting point based on criteria such as information gain or variance reduction.

Key Characteristics



Recursive splitting mechanism

Starting from the root node, maximize segmentation purity based on feature thresholds (such as minimizing mean square error) until stopping conditions are met (such as maximum depth or minimum sample size).

Leaf node prediction

Each leaf node stores the mean or median of the target variable as the predicted value for the samples in that region.

Overfitting risk

If the tree depth is too deep, it will capture noisy data, which requires pruning (such as cost complexity pruning) or setting early stopping conditions to optimize generalization ability.

Applications in Real-World Problems

01

House price prediction

Construct a regression tree using characteristics such as house area and location to predict continuous house prices, adapting to market fluctuations and non-linear trends.

02

Sales trend analysis

Predicting future sales revenue through historical sales data (such as seasons and promotional activities) to assist inventory management decisions.

03

Medical prognosis assessment

Based on patient clinical indicators such as age and biochemical parameters, predict the risk of disease progression and support personalized treatment.

04

Industrial quality control

monitoring the correlation between production parameters (such as temperature and pressure) and product quality, optimizing manufacturing processes.

02

Core Principles of Decision Tree Regression

Tree Structure and Components



Hierarchical decision-making mechanism

The decision tree achieves data segmentation through a hierarchical structure of root nodes, internal nodes, and leaf nodes, where each internal node represents a judgment condition for a feature attribute. This structure intuitively displays the decision path from input to output.



Dynamic space partitioning

The feature space is partitioned into rectangular regions using recursive binary partitioning, with each region corresponding to a leaf node. This non parametric method can automatically adapt to the nonlinear relationship of data without the need for preset function forms.



Strong interpretability

Compared with black box models, the tree structure of decision trees can be directly visualized, and decision rules can be explained through "if then" statements, which is particularly suitable for application scenarios that require model transparency.

Splitting Criteria (e.g., MSE Reduction)

Feature selection algorithm

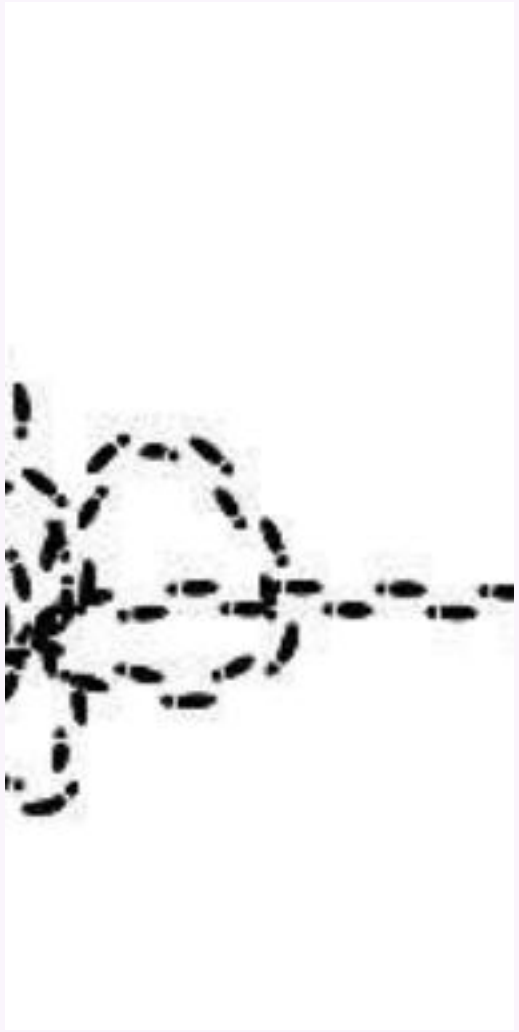
Traverse all features and their possible splitting thresholds, calculate the sum of MSE of the left and right subsets after splitting, and select the feature threshold combination that maximizes the decrease in total MSE as the splitting point.

Greedy search strategy

using a greedy algorithm with local optima to optimize and split layer by layer. Although it cannot guarantee global optimality, it has high computational efficiency and is suitable for processing large-scale datasets.

Stop condition control

Prevent overfitting by setting hyperparameters such as maximum depth and minimum leaf sample size, and stop splitting when the node sample size is less than the threshold or the MSE does not decrease significantly.



Leaf Node Prediction (Mean Value)

- **Predicted Value Calculation**: The predicted value at a leaf node is determined by averaging the target values of all training samples that fall into that node. This method ensures robustness against outliers and maintains a low computational complexity of O(1).
- **Mathematical Expression**: For a leaf node with n samples, the predicted value is calculated as $\hat{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$. This calculation aims to minimize the squared error within the node.
- **Stepped Prediction Function**: All samples within the same leaf node receive the same predicted value, leading to a stepwise prediction function. While increasing the tree depth can enhance prediction accuracy, it is crucial to balance model complexity and generalization ability.
- **Prediction Anomalies and Ensemble Methods**: Prediction anomalies can occur at the edges of the feature space. Ensemble techniques like random forests mitigate this issue by averaging predictions across multiple trees.

Decision Tree Regression: Formulas

1. **Prediction:**

$$\hat{y} = \frac{1}{n_{\text{leaf}}} \sum_{i \in \text{leaf}} y_i$$

Where n_{leaf} = samples in leaf node.

2. **Splitting Criterion (Variance Reduction):**

$$\Delta = V_{\text{parent}} - \left(\frac{n_{\text{left}}}{n_{\text{parent}}} V_{\text{left}} + \frac{n_{\text{right}}}{n_{\text{parent}}} V_{\text{right}} \right)$$

- $V = \frac{1}{n} \sum (y_i - \bar{y})^2$ (variance)
- Split chosen to **maximize** Δ (minimize child variance).

Key Concepts

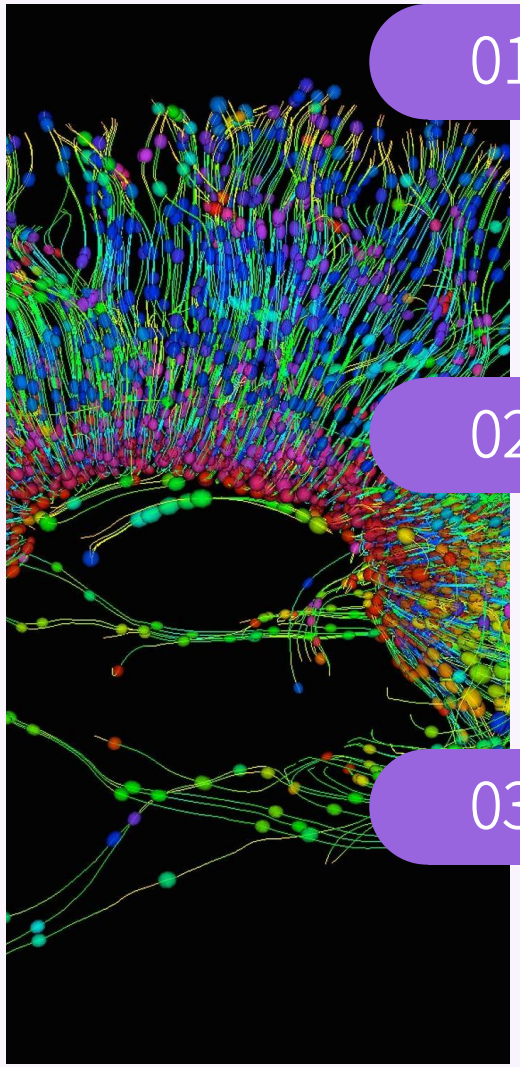
1. **Splitting Logic**:
   - Iteratively split data to minimize **variance** in child nodes.
   - Stop when `max_depth` is reached or nodes are "pure" (variance ≈ 0).
2. **Prediction Nature**:
   Outputs are **piecewise constant functions** (horizontal lines in partitions).
3. **Trade-offs**:
   - Shallow trees (**low depth**): Underfit (high bias).
   - Deep trees (**high depth**): Overfit noise (high variance).

💡 **Practical Tip**: Tune `max_depth` and `min_samples_leaf` using cross-validation to balance bias-variance.

03

Model Construction Process

Data Preparation and Feature Selection



01

Data cleaning and standardization

Decision tree regression is sensitive to missing and outlier values, and requires filling, deleting, or interpolating missing data, as well as standardizing continuous features (such as Z-score or Min Max) to improve model stability.

02

Feature correlation analysis

By calculating Pearson correlation coefficient or mutual information, select features strongly correlated with the target variable, eliminate redundant features to reduce the risk of overfitting, and retain variables with strong interpretability.

03

Category variable encoding

If the data contains categorical features such as color and region, one hot encoding or target encoding should be used to convert them into numerical form to ensure that the algorithm can handle them.

Recursive Partitioning Methodology

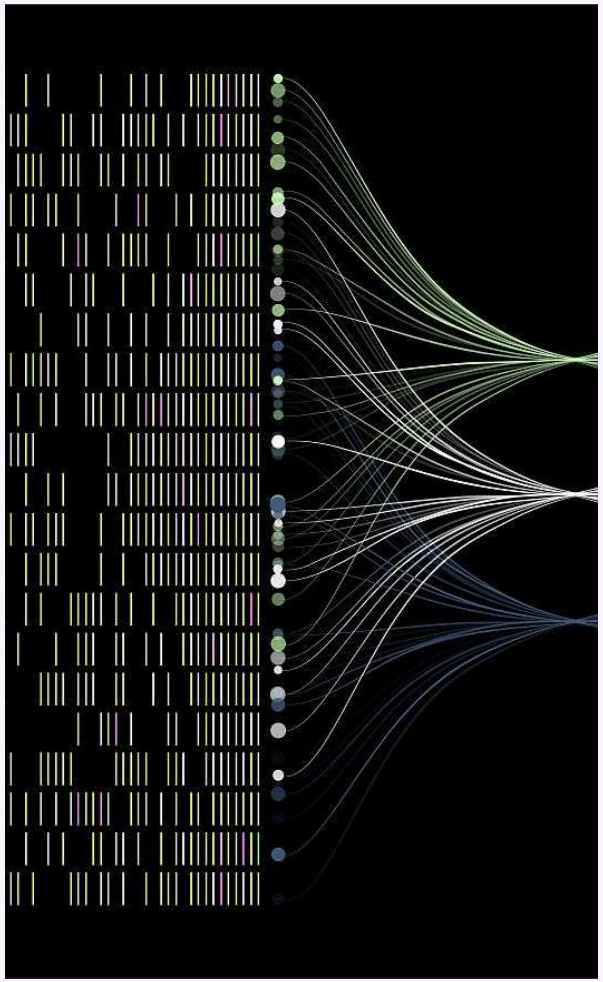
- Greedy splitting strategy: During each split, the algorithm traverses all features and possible splitting points (such as mean and median), and selects the optimal splitting point that minimizes the mean squared error (MSE) or mean absolute error (MAE).

- Maximizing Information Gain: By calculating the information gain (or Gini impurity reduction) before and after splitting, priority is given to selecting features that can minimize the impurity of child nodes, ensuring that each split effectively improves prediction accuracy.

- Local optimal solution problem: Due to recursive splitting only considering the optimal solution of the current node, it may lead to a global suboptimal tree structure, which can be alleviated through ensemble methods such as random forests.

- Optimization of computational efficiency: For large-scale datasets, approximate splitting algorithms (such as histogram approximation) or parallel computing are used to accelerate the feature search process, balancing accuracy and training speed.

Stopping Conditions (Depth, Samples per Leaf)



Maximum tree depth limit

Control the maximum level of the tree by setting the 'max_depth' parameter to prevent overfitting of the model; If the depth is too high, it will capture noise; if it is too low, it will be underfitting and requires cross validation parameter tuning.

Minimum sample size for leaf nodes

Set the 'min_Samples_leaf' threshold (such as 5 or 10) to ensure that each leaf node contains enough samples to make stable predictions and avoid high variance caused by too few samples.

Splitting gain threshold

If the information gain after splitting is lower than the preset value (such as 0.01), the splitting is terminated to prevent irrelevant features from making small but meaningless improvements to the model.

Numeric Example: House Price Prediction

Dataset:

Rooms (X)	Price (y)
1	100
2	150
3	175
4	200
5	300

Mathematical Foundations: Prediction & Impurity

Prediction Formula

$$\hat{y}_{\text{leaf}} = \frac{1}{n_{\text{leaf}}} \sum_{i \in \text{leaf}} y_i$$

Every observation falling into the same leaf receives the **mean** of training targets in that leaf.

Variance Measure

$$V = \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y})^2$$

Intuition: Lower variance ⇒ more homogeneous node ⇒ better explanation of target variability.

Stopping Conditions (preview)

- Max depth
- Min samples per leaf

Splitting Criterion: Maximizing Variance Reduction

Objective

Choose the split that **maximizes**:

$$\Delta = V_{\text{parent}} - \left(\frac{n_{\text{left}}}{n_{\text{parent}}} V_{\text{left}} + \frac{n_{\text{right}}}{n_{\text{parent}}} V_{\text{right}} \right)$$

Interpretation of Δ

- Δ = amount of **uncertainty removed** by the split
- Larger Δ ⇒ more informative split

Computational Workflow

1. Enumerate all candidate thresholds
2. Compute weighted child variances
3. Select threshold with highest Δ

Step 1: Root Node Impurity

- **Samples**: [100, 150, 175, 200, 300]
- **Mean**: $\bar{y} = \frac{100+150+175+200+300}{5} = 185$
- **Variance**:

$$V = \frac{(100-185)^2 + (150-185)^2 + \cdots + (300-185)^2}{5} = \frac{7225 + 1225 + 10(}{5}$$

Step 2: Evaluate Splits

Candidate 1: Split at $X \leq 1.5$

- **Left** $(X = 1)$: [100]

$$V_{\text{left}} = 0, \quad n_{\text{left}} = 1$$

- **Right** $(X \geq 2)$: [150, 175, 200, 300]

$$\bar{y}_{\text{right}} = 206.25, \quad V_{\text{right}} = \frac{(150-206.25)^2 + \cdots + (300-206.25)^2}{4} = 3,242.1$$

- **Variance Reduction**:

$$\Delta_1 = 4,400 - \left(\frac{1}{5} \times 0 + \frac{4}{5} \times 3,242.19\right) = 4,400 - 2,593.75 = \boxed{1,806.25}$$

Candidate 2: Split at $X \leq 2.5$

- **Left** ($X \leq 2$): [100, 150]

$$\bar{y}_{\text{left}} = 125, \quad V_{\text{left}} = \frac{(100-125)^2 + (150-125)^2}{2} = 625$$

- **Right** ($X \geq 3$): [175, 200, 300]

$$\bar{y}_{\text{right}} = 225, \quad V_{\text{right}} = \frac{(175-225)^2 + (200-225)^2 + (300-225)^2}{3} = 2,916.$$

Candidate 3: Split at $X \leq 4.5$

- **Left** ($X \leq 4$): [100, 150, 175, 200]

$$\bar{y}_{\text{left}} = 156.25, \quad V_{\text{left}} = \frac{(100 - 156.25)^2 + \cdots + (200 - 156.25)^2}{4} = 1,367.19$$

- **Right** ($X = 5$): [300]

$$V_{\text{right}} = 0, \quad n_{\text{right}} = 1$$

- **Variance Reduction**:

$$\Delta_3 = 4,400 - \left(\frac{4}{5} \times 1,367.19 + \frac{1}{5} \times 0\right) = 4,400 - 1,093.75 = \boxed{3,306.25}$$

Step 3: Select Best Split

Max $\Delta = 3,306.25$ at $X \leq 4.5 \rightarrow$ **Optimal split.**

- **Left leaf prediction:** $\hat{y} = 156.25$ (mean of [100,150,175,200])
- **Right leaf prediction:** $\hat{y} = 300$

Prediction for New Data

House (Rooms)	Path	Predicted Price
3	$X \leq 4.5 \rightarrow$ Left	156.25
6	$X > 4.5 \rightarrow$ Right	300

Key Insights

1. **Splits minimize weighted variance** in child nodes.
2. **Predictions are constants** within each leaf.
3. **Deep trees** capture complex patterns but risk overfitting (e.g., splitting noisy data).

> 💡 **Practical Tip**: Use `min_samples_leaf=5` to avoid leaves with <5 samples.

04

Advantages and Limitations

Interpretability and Non-Linearity Handling

01

Intuitive visualization

The decision tree displays the decision path through a tree structure, and the splitting conditions (such as feature thresholds) of each node and the predicted values of leaf nodes can be directly explained, making it suitable for conveying model logic to non-technical personnel.

02

Automatic feature interaction

Decision trees can automatically capture non-linear relationships between features (such as the combination condition of "age>30 and income<50k"), without the need for manual construction of interaction terms, making them suitable for complex data patterns.

03

No data preprocessing required

insensitive to missing values, outliers, and dimensional differences, can directly handle mixed type data (such as numerical and categorical features), simplifying the modeling process.

Overfitting Risks

High variance problem

Decision trees tend to overfit the noise in the training data, especially for deep trees, resulting in a significant decrease in performance on the test set and poor generalization ability.

Data fragmentation

As the tree depth increases, the sample size of nodes gradually decreases, and fragmentation may be based on statistical randomness (such as a few abnormal samples), resulting in meaningless rules.

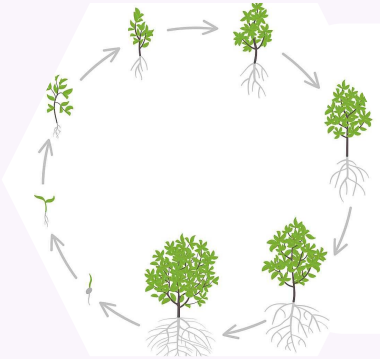
Local optimal splitting

Greedy algorithms only consider the optimal splitting of the current node, which may ignore the globally optimal segmentation scheme, leading to the model falling into suboptimal solutions.

Sensitive to minor changes

Slight perturbations in training data (such as deleting some samples) may lead to completely different tree structures and lower stability.

Mitigation Strategies (Pruning, Regularization)

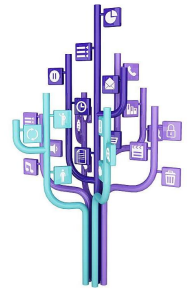


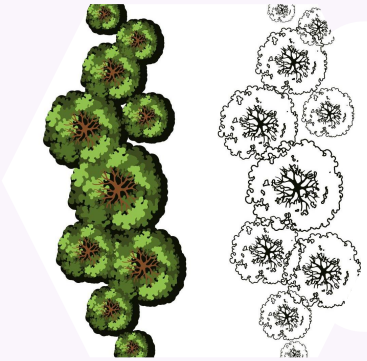
Post Pruning

Allow the tree to fully grow first, and then remove subtrees that do not contribute to generalization performance through cross validation (such as cost complexity pruning), balancing complexity and accuracy.

Pre pruning

Restricting the growth conditions of the tree during training (such as maximum depth, minimum sample size of leaf nodes, and splitting gain threshold) to prevent premature overfitting of the model.



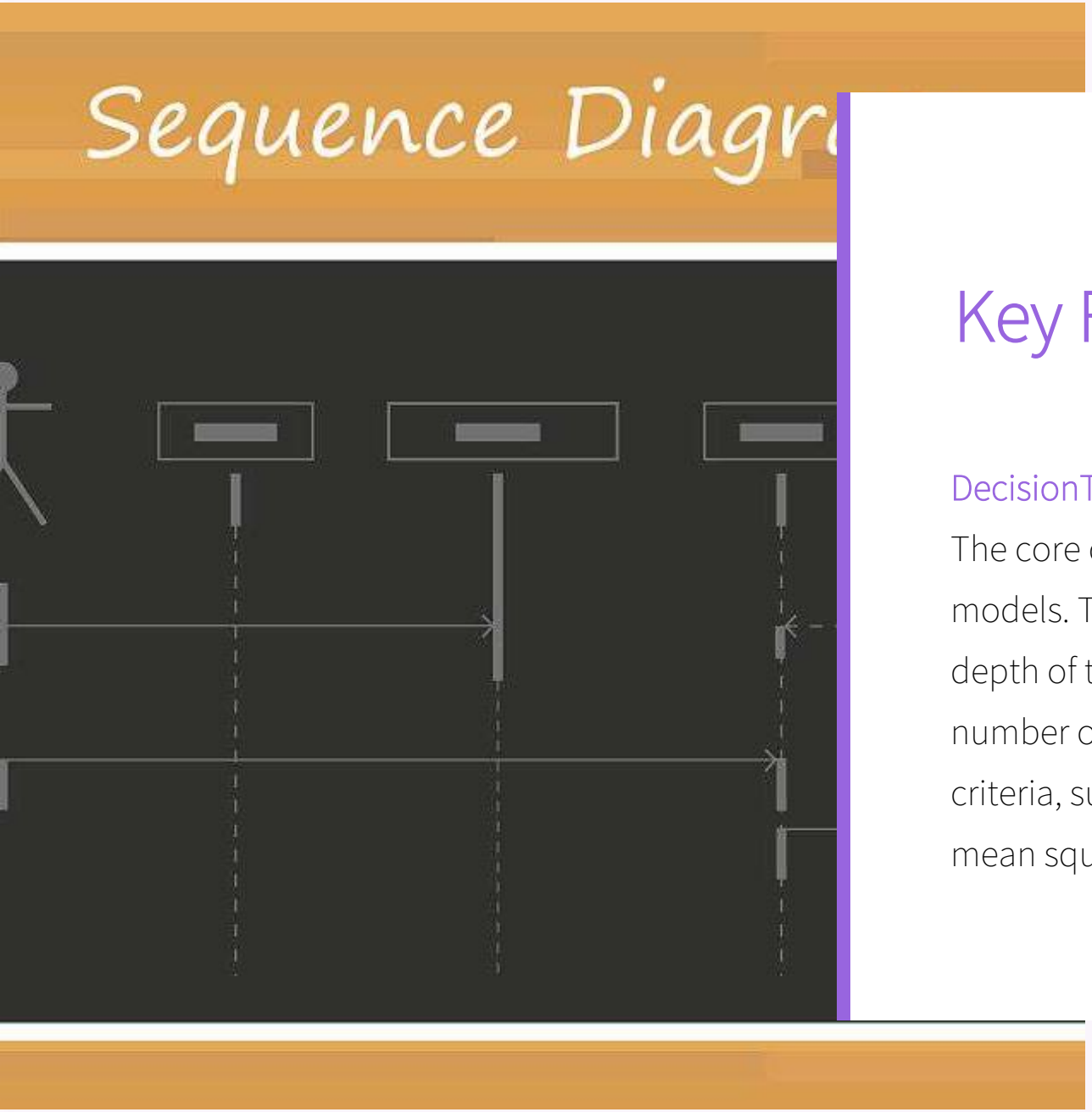


Regularization technique

Introducing penalty terms (such as the complexity parameter α of CART algorithm) to balance the goodness of fit and model complexity in the loss function, forcing the generation of a more concise tree structure.

05

Implementation with Scikit-Learn (Python)



Key Functions and Parameters

DecisionTreeRegressor()

The core class in Scikit Learn used to initialize decision tree regression models. The key parameters include 'max_depth' (controlling the maximum depth of the tree to prevent overfitting), 'min_Samples_split' (the minimum number of samples required for node splitting), and 'criteria' (splitting criteria, such as' mse 'mean square error or' friedmanmse 'improved version mean square error).

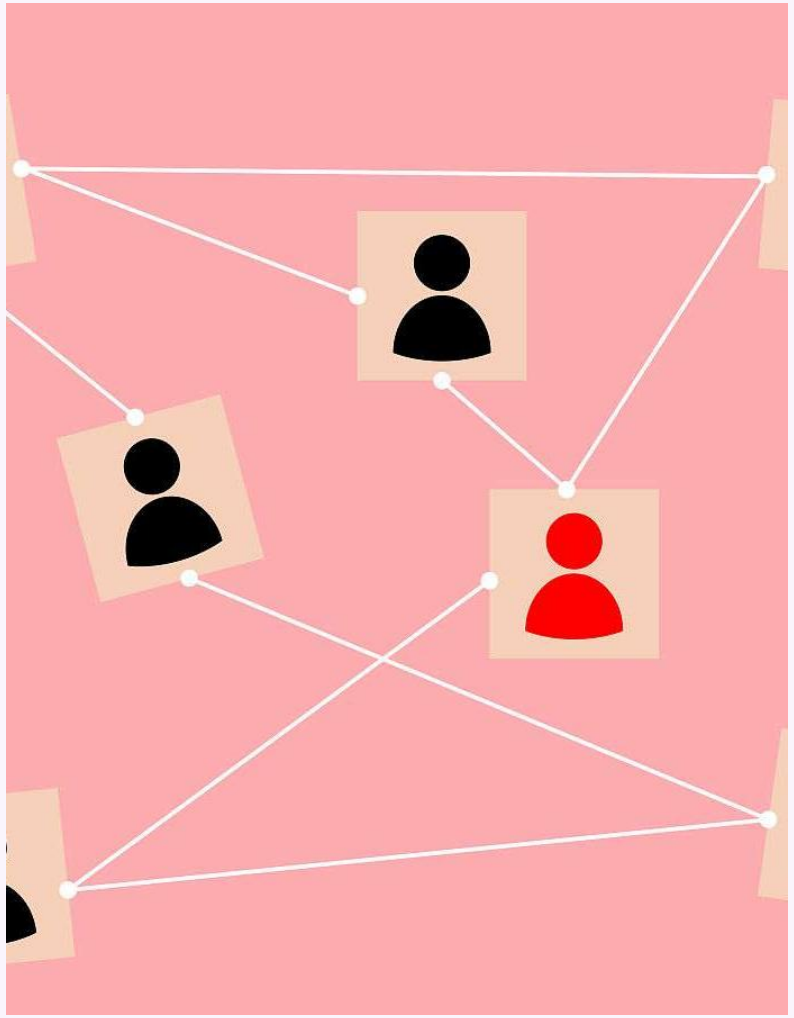
Key Functions and Parameters

Fit (X, y)

Training method, input feature matrix 'X' and target variable 'y', construct decision tree model. It is necessary to ensure that the data has no missing values and that the features are numerical or encoded categorical variables.

Predict (X_test)

a prediction method that takes the test set feature 'X_test' as input and outputs continuous predicted values. The model uses the mean of the target variable at the leaf nodes as the prediction result.



Example Code Walkthrough

- **Data Generation and Preparation**:
  - Utilize `numpy.random` to create random data.
  - Example: `X = np.sort(5 * np.random.rand(80, 1), axis=0)`.
  - Add noise to the target variable `y` to simulate real-world scenarios.
  - Example: `y[::5] += 3 * (0.5 - np.random.rand(16))`.
- **Model Training**:
  - Instantiate a `DecisionTreeRegressor` with a specified `max_depth`.
  - Example: `DecisionTreeRegressor(max_depth=2)`.
  - Train the model using `fit(X, y)`.
  - Adjust `max_depth` to observe fitting effects, such as underfitting with small depth and overfitting with large depth.
- **Visualizing the Decision Tree**:
  - Use `sklearn.tree.plot_tree` to visualize the tree structure.
  - Employ `matplotlib` to display predicted values at split nodes, thresholds, and leaf nodes.
- **Prediction and Plotting**:
  - Generate test data: `X_test = np.arange(0.0, 5.0, 0.01)[:, np.newaxis]`.
  - Call `predict(X_test)` to obtain predictions.
  - Plot the prediction curve and compare it with the distribution of real data.

Model Evaluation Metrics (MSE, R^2)

Mean Square Error (MSE)

Calculate the mean square difference between the predicted value and the true value, reflecting the absolute magnitude of the model error. The formula is MSE=1/n Σ (y_true - y_pred) ^ 2, where smaller values indicate higher model accuracy.

R^2 coefficient of determination

measures the proportion of variance explained by the model to the target variable, with a range of [0,1]. The formula is R^2=1- (Σ (y_true - y_pred) ^ 2/Σ (y_true - y_mean) ^ 2), the closer it is to 1, the better the fitting effect.

Cross validation

Use 'sklearn. model_delection. cross_ra_score' combined with MSE or R^2 to evaluate the model's generalization ability and avoid bias in single data partitioning.

Example: Fitting a Sine Wave

Goal: Model noisy sine wave data using scikit-learn

Step 1: Generate Data

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor

# Generate 80 random points in [0, 5]
X = np.sort(5 * np.random.rand(80, 1), axis=0)
# Create sine wave target + add noise
y = np.sin(X).ravel()
y[::5] += 3 * (0.5 - np.random.rand(16))  # Add noise to every 5th point
```

Step 2: Train Models

```
# Train two models with different depths
regr_2 = DecisionTreeRegressor(max_depth=2)  # Simple model (4 leaf nodes)
regr_5 = DecisionTreeRegressor(max_depth=5)  # Complex model (up to 32 leaf nodes)
regr_2.fit(X, y)
regr_5.fit(X, y)
```

copy

Step 3: Predict & Visualize

```
# Create test data
X_test = np.arange(0.0, 5.0, 0.01)[:, np.newaxis]
y_pred_2 = regr_2.predict(X_test)  # Predictions (max_depth=2)
y_pred_5 = regr_5.predict(X_test)  # Predictions (max_depth=5)

# Plot results
plt.figure()
plt.scatter(X, y, s=20, edgecolor="black", label="Noisy Data")
plt.plot(X_test, y_pred_2, color="red", label="max_depth=2")
plt.plot(X_test, y_pred_5, color="green", label="max_depth=5")
plt.xlabel("X"); plt.ylabel("y"); plt.legend()
plt.title("Decision Tree Regression: Sine Wave Fitting")
plt.show()
```

06

Practical Case Studies

Predicting Housing Prices



01

The importance of feature selection

In house price prediction, key features such as room quantity, geographical location, crime rate, etc. have a significant impact on model performance. Evaluating feature importance through information gain or Gini coefficient can optimize model accuracy.

02

Data preprocessing

It is necessary to handle missing values (such as filling in the mean) and outliers (such as the 3 σ principle), and standardize or normalize numerical features (such as area) to ensure model stability.

03

Tree depth control

Trees that are too deep can easily lead to overfitting, and cross validation is needed to select the optimal depth (such as limiting max_depth=5) to balance bias and variance.

Predicting Housing Prices



Model evaluation metrics

commonly used mean square error (MSE), mean absolute error (MAE), and R^2 score to evaluate performance. MSE is sensitive to outliers, while MAE is more robust.



Interpretability advantage

Decision trees can intuitively display feature splitting logic (such as "If the number of rooms is greater than 3, the housing price is high"), making it easy to explain the prediction results to non-technical personnel.

Sales Forecasting

- Seasonal feature processing: It is necessary to introduce temporal features (such as months and quarters) or Fourier transform to capture the periodic patterns of sales data (such as holiday peaks).

- Multivariate collaborative analysis: Combining external variables such as promotional activities and competitor pricing, improving prediction accuracy through feature crossover (such as "discount rate x advertising investment").

- Dynamic parameter tuning strategy: Use GridSearchCV to optimize hyperparameters (such as min_stamples_split) to adapt to the sales model differences of different product lines.

- Business scenario adaptation: For fast-moving consumer goods (high-frequency low profit) and durable goods (low-frequency high profit), it is necessary to adjust the weight of the loss function (such as focusing on predicting high-value orders).

Energy Consumption Estimation

1 Nonlinear relationship modeling

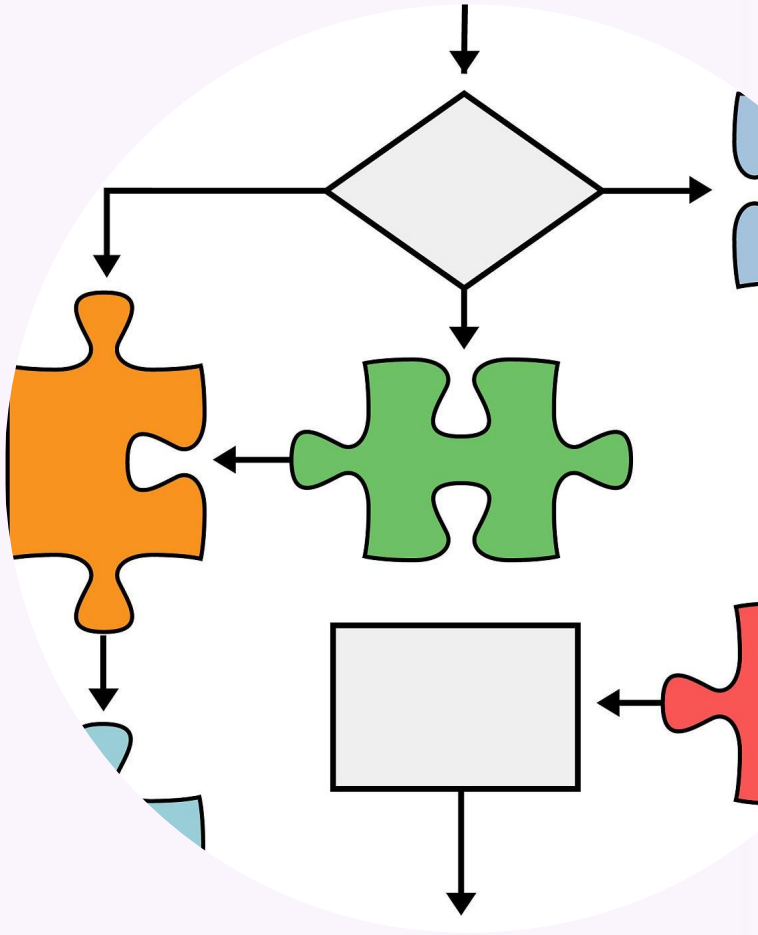
Decision trees are naturally suitable for handling the nonlinear relationship between electricity consumption, temperature, and humidity, without the need for manually constructing polynomial features.

2 Time series slicing

Segment historical energy consumption data by hour/day, construct sliding window features (such as the average of the past 7 days), and enhance the ability to capture temporal dependencies.

3 Abnormal energy consumption detection

Identifying outliers (such as sudden increases in nighttime electricity consumption) through decision tree splitting thresholds to assist in fault diagnosis or energy efficiency optimization.



01 Decision tree rules optimized by LLM



Rule analysis and reconstruction

01

Rule semantic parsing

Utilizing the natural language understanding ability of LLM, deep semantic analysis is performed on the original rules generated by decision trees to identify redundant, conflicting, or fuzzy logical branches and reconstruct them into a more concise set of rules.

02

Context correlation optimization

Capture implicit correlations between rules through LLM, such as merging similar conditions (such as the cross influence of "age<30" and "income>50k"), to generate more generalizable composite rules.

03

Dynamic weight adjustment

Based on LLM evaluation of data distribution, dynamic weights are assigned to features in decision tree rules, prioritizing the preservation of high discriminative features (such as nodes with information gain rate>0.3).

Rule analysis and reconstruction



Enhanced interpretability

Using LLM to generate natural language descriptions of rules, such as converting binary splitting conditions into business friendly expressions like "recommending promotional strategies when user activity is below 5 times per week".



Abnormal rule filtering

Identify rules with low support (such as<5%) or high conflict probability through LLM, and remove them by combining statistical significance test (p-value<0.01).

Statistical similarity enhancement

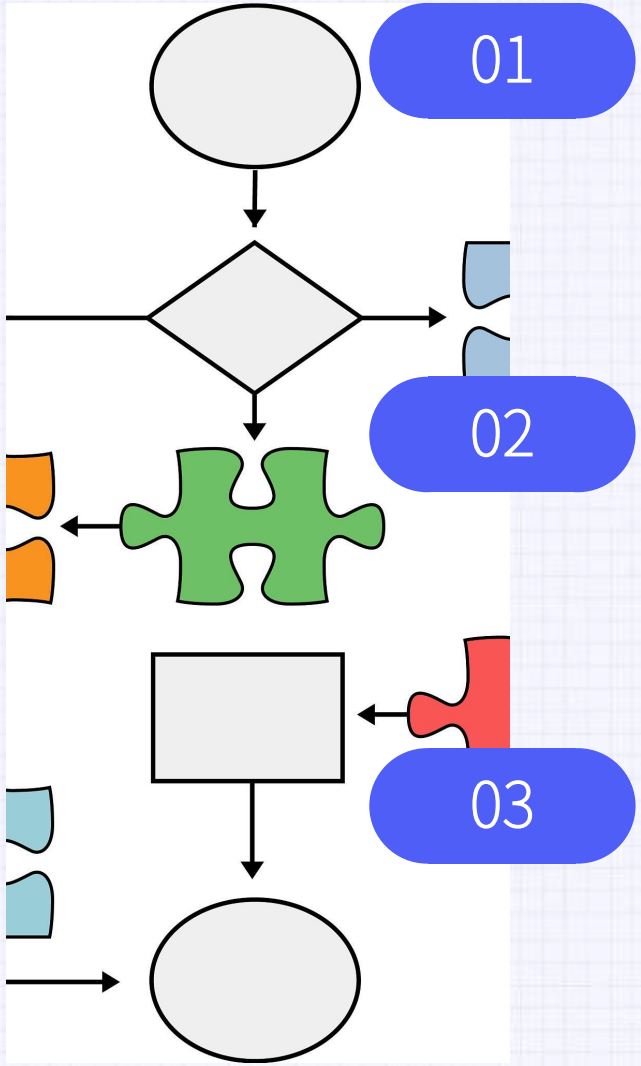
- Distribution alignment technique: using LLM to extract potential feature distributions of training data (such as user behavior clustering), forcing decision trees to generate consistency rules on subsets of similar statistical characteristics.

- Cross dataset generalization: By comparing the statistical differences (such as KL divergence) between the source domain and the target domain through LLM, adjust the decision tree splitting threshold to improve transfer performance.

- Improvement of noise robustness: Utilizing the adversarial sample generation capability of LLM, simulating input disturbances (such as ± 10% feature shift), optimizing the decision tree's tolerance to noise (such as F1 fluctuation<2%).

- Long tail data compensation: For sparse categories (such as rare disease diagnosis), LLM guides decision trees to adopt cost sensitive splitting (such as class weights inversely proportional to frequency), balancing recall and accuracy.

Improvement without domain dependency



Universal feature encoding

LLM automatically converts raw features (such as text and time-series data) into numerical representations that can be processed by decision trees (such as BERT embedded PCA dimensionality reduction), eliminating the need for manual feature engineering.

Self supervised rule generation

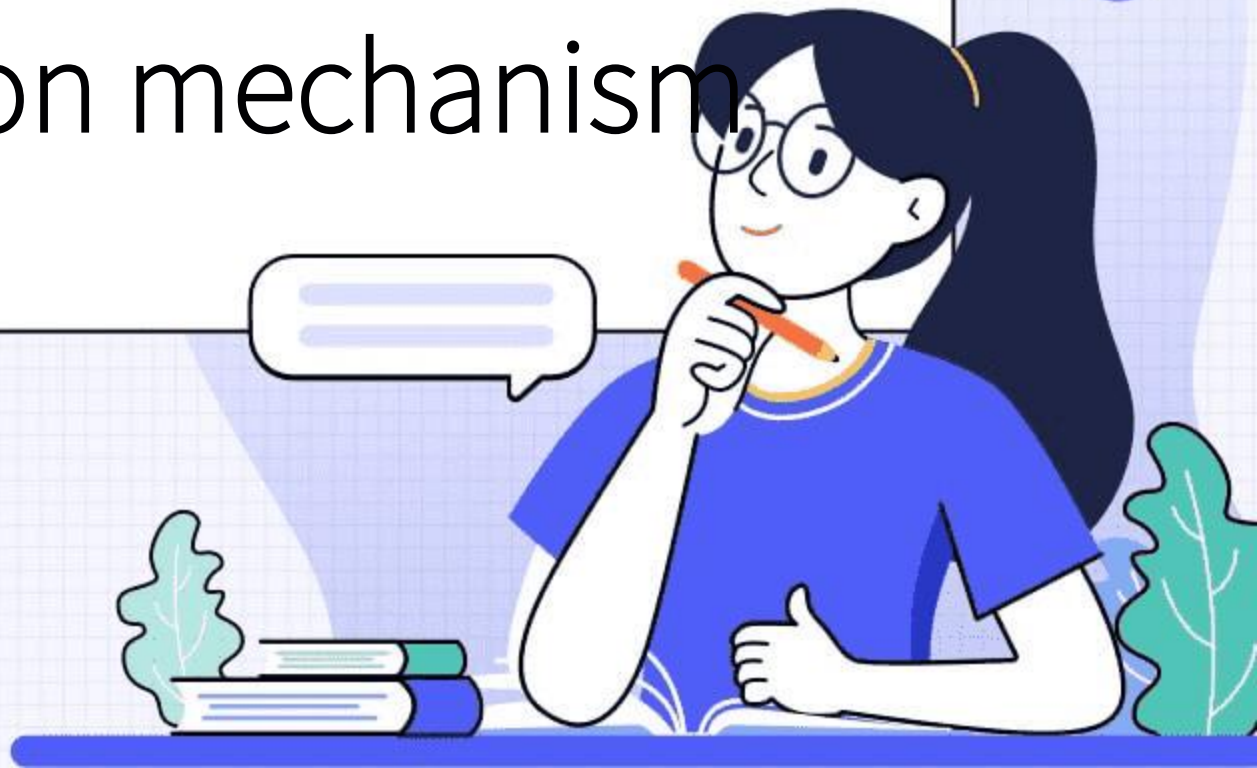
Using the zero sample inference capability of LLM, potential rules (such as clustering center distance threshold) are derived from unlabeled data as weakly supervised signals for decision tree initialization.

Cross modal knowledge fusion

Integrating the multimodal understanding of LLM (such as image description generation) with structured reasoning of decision trees to achieve mixed modal decision-making (such as joint diagnosis of "CT image features+patient medical history").

02

Rule based error correction mechanism



Residual error prediction



Error source analysis

Identify systematic biases in the joint prediction of LLM and decision trees through statistical models, such as high-frequency error types (such as semantic ambiguity or data sparsity problems in specific domains), quantify residual distributions to locate the main sources of errors.



Dynamic compensation model

Build lightweight residual predictors (such as linear regression or shallow neural networks) to correct the output bias of the main model in real time, especially suitable for error compensation of long tail data or low confidence samples.



Iterative optimization framework

Feedback the residual prediction results to the training stage, adjust the weight allocation of LLM and decision tree through adversarial training or reinforcement learning, and form a closed-loop system with gradually decreasing errors.

Gradient network calibration

- Gradient alignment technique: Design cross model gradient propagation paths to ensure that the attention mechanism of LLM and the splitting criteria of decision trees are optimized synchronously during backpropagation, avoiding parameter update conflicts caused by differences in learning rates.

- Temperature scaling strategy: Calibrate the temperature parameters of the category probabilities output by the decision tree to match the Softmax distribution scale of LLM, reducing the ensemble error caused by incompatible confidence levels between the two models.

- Adversarial perturbation suppression: Introducing an adversarial sample detection module in gradient updates to filter out abnormal gradient directions that may cause overfitting noise or LLM illusion generation in decision trees.

- Hierarchical gradient fusion: Divide the gradient fusion area according to the depth of the network, with shallow parameters focusing on feature extraction consistency and deep parameters focusing on task specific optimization to enhance the generalization ability of the joint model.

Methods for improving accuracy

Hybrid feature engineering

Combining the hidden embedding of LLM (such as BERT's [CLS] vector) with the explicit rule features of decision trees, the optimal feature combination is dynamically selected through gating mechanisms to enhance the model's ability to capture complex patterns.

Uncertainty aware ensemble

Quantify the prediction uncertainty of two types of models based on Bayesian framework, and initiate a secondary inference process for high uncertainty samples (such as Monte Carlo Dropout sampling or decision tree multi-path voting).

Domain adaptive training

Adopting a curriculum learning strategy, the initial output of LLM is first constrained by strong decision tree rules, and then the model degrees of freedom are gradually released to achieve a smooth transition from rule driven to data-driven.

03 Construction of zero sample decision tree



Probability estimation and split generation

01

Probability distribution modeling

LLM is used to model the potential features of input data, utilizing its powerful contextual understanding ability to generate conditional probability tables for different feature combinations, providing data support for decision tree splitting.

02

Dynamic splitting strategy

Combining the probability confidence level output by LLM, dynamically adjusting the splitting threshold of the decision tree, prioritizing the feature with the highest information gain for node splitting, and improving the model's generalization ability in zero sample scenarios.

03

Multimodal feature fusion

For mixed type data such as text, numerical values, and categories, LLM embedding is used to represent a unified feature space, and then probability weighted splitting rules are generated to solve the limitations of traditional decision trees in processing heterogeneous data.

Minimize Gini impurity

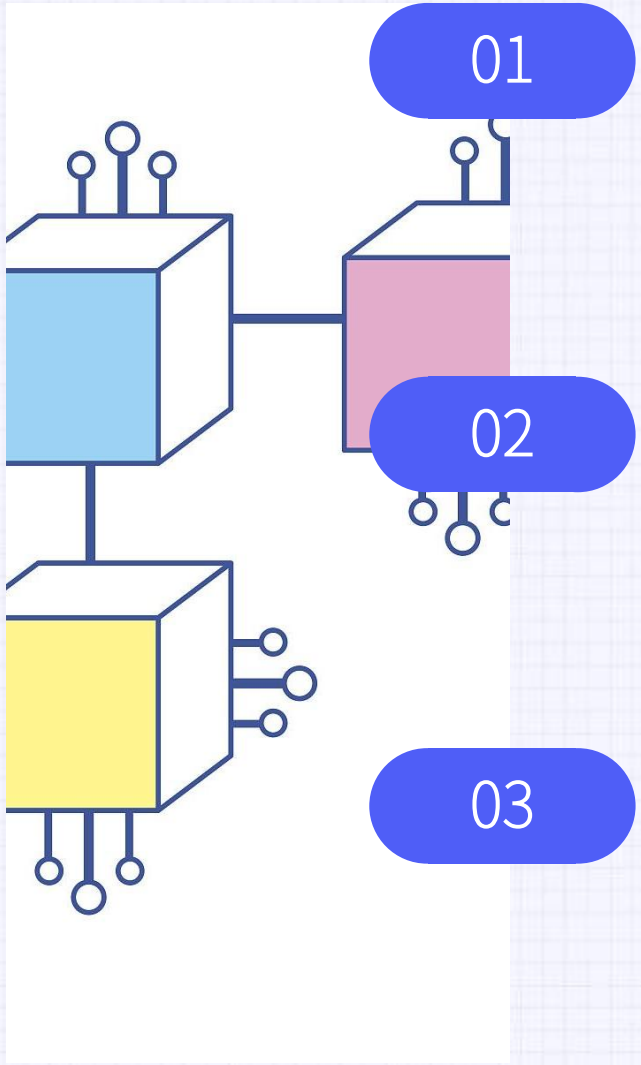
- Weighted Gini coefficient optimization: Under zero sample conditions, the Gini coefficient calculation method is redefined based on LLM prediction of category probability, and a sample weight adjustment mechanism is introduced to reduce the impact of noisy data on the selection of splitting points.

- Local optimal suppression: Simulate the Gini coefficient changes of different splitting paths through Monte Carlo sampling, combined with gradient descent to avoid falling into local optima and ensure the stability of the global splitting strategy.

- Category imbalance correction: For long tail distribution data, the prior probabilities generated by LLM are used to modify the Gini coefficient formula, giving higher weights to minority categories and improving the model's ability to recognize rare samples.

- Cross validation of split points: In an unlabeled data environment, virtual labels are generated using LLM and k-fold cross validation is used to evaluate the effectiveness of split points and prevent overfitting issues.

Unsupervised tree structure generation



01

Self supervised node generation

Utilizing the masking language modeling capability of LLM, random masking is applied to input features and reconstruction errors are predicted. High error regions are selected as candidate splitting nodes to achieve unsupervised tree structure initialization.

02

Hierarchical clustering guidance

Using **semantic embedding vectors extracted by LLM for hierarchical clustering,** mapping the clustering results to the hierarchical relationship of the decision tree, and automatically determining the depth and number of branches of the tree.

03

Adversarial regularization

Introducing the collaborative training of Generative Adversarial Networks (GANs) and LLMs, optimizing the sparsity of decision tree structure through discriminator feedback, and avoiding the generation of redundant or overly complex subtrees.

04 Interpretability enhancement technology



Natural language interpretation generation



Improving model comprehensibility

Using natural language generation technology to transform complex model decision-making processes into human readable textual descriptions, helping non-technical users understand model logic and lowering the threshold for AI applications.



Enhance decision credibility

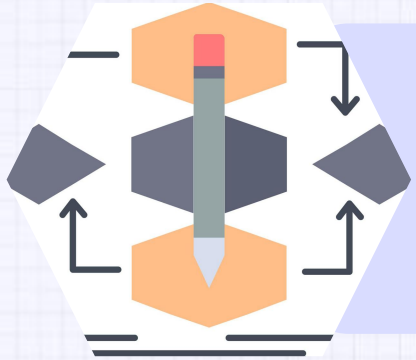
Clear explanations can verify whether the model inference conforms to domain common sense or business rules, providing traceable basis for key decisions such as medical diagnosis and financial risk control.



Support rapid iterative optimization

Intuitive explanations can help developers locate model biases or error patterns, and adjust training data or algorithm architecture accordingly.

Principle of GPTree Implementation

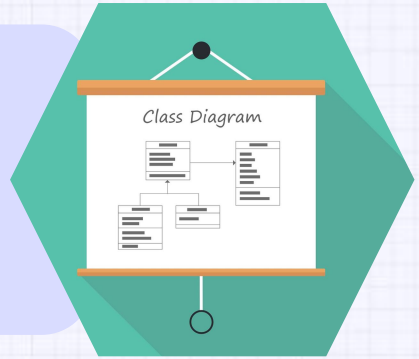


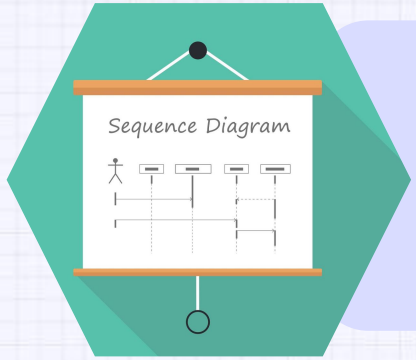
Dynamic feature interaction analysis

Using LLM to extract potential features from text/unstructured data, decision trees generate interpretable branch rules based on these features, forming a "feature rule" mapping table.

Joint training mechanism

By alternating optimization strategies, the embedding layer of LLM and the node splitting criteria of decision tree are synchronously trained to ensure consistency between the language model output and the decision tree logic.





Multi granularity interpretation generation

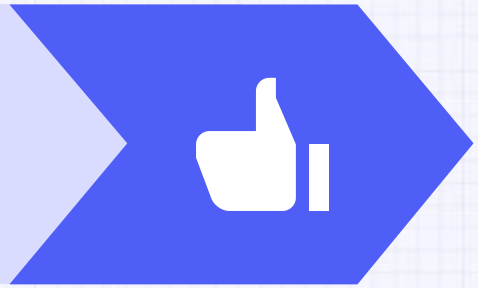
Automatically adjust the depth of interpretation according to user needs, such as providing feature importance ranking for expert users and outputting case analogy explanations for ordinary users.

Transparent decision-making path display



Using tree diagrams and highlighting techniques to visually display the complete path of the sample from the root node to the leaf node, annotating key splitting thresholds and feature contributions.

Support interactive exploration, allowing users to click on any node to view detailed segmentation criteria (such as "triggering risk control rules when user age>30 and transaction frequency<5 times/week").





Synchronize the generation of decision basis summaries during model runtime, such as "Refusing loan applications due to: 1) historical overdue times ⩾ 3 times; 2) income liability ratio exceeding 85%".

Transparent decision-making path display

Provide counterfactual explanation function (such as "if annual income increases by 20000 yuan, the probability of passing will increase to 72%") to assist users in understanding improvement directions.

For multimodal input data such as images/text, structured features are processed using decision trees, natural language descriptions are generated using LLM, and finally integrated into a unified report.

Example: In medical imaging diagnosis, the decision tree outputs quantitative indicators such as "nodule diameter>1.5cm", and LLM supplements semantic descriptions such as "edge spiciness".

05 Hybrid architecture and symbolic reasoning



Design of Multi Agent Systems

- Distributed Collaboration Framework: Multi Agent Systems (MAS) achieve collaborative optimization between LLM and decision trees through a distributed architecture. Each agent is responsible for specific tasks such as data preprocessing, feature extraction, or model inference, and shares information through message passing mechanisms (such as MCP) to improve overall efficiency.

- Dynamic role allocation: Agents can dynamically switch roles based on task requirements, such as generating candidate solutions through LLM during the inference phase, and performing logical verification through decision tree agents to form complementary decision loops.

- Fault tolerance and adaptability: The system design needs to integrate redundancy mechanisms and real-time monitoring modules. When an intelligent agent fails, other nodes can take over the task, while optimizing the cooperation strategy between intelligent agents through reinforcement learning.

Combining rule-based reasoning and induction

- Rule guided model training: Encode expert knowledge into rule constraints of decision trees (such as clinical diagnosis and treatment guidelines), which are used to screen candidate answers generated by LLM, reduce hallucination output, and improve the interpretability of results.

- Hybrid reasoning pipeline: LLM processes unstructured text input (such as patient complaints), outputs structured features, and hands them over to decision trees for logical reasoning, achieving a transition from fuzzy semantics to precise classification.

- Bidirectional knowledge distillation: Through a joint training framework, the decision tree learns implicit patterns (such as rare case associations) from LLM, while LLM absorbs explicit rules (such as diagnostic thresholds) from the decision tree, improving the generalization ability of both parties.

- Real time feedback loop: The classification results of the decision tree can be used to reverse the generation process of LLM (such as adjusting keyword weights through attention mechanisms), forming an iterative optimization inference chain.

Clinical Decision Support Application



Hierarchical diagnosis system

LLM preliminary analysis of free text in electronic medical records (such as symptom descriptions), decision trees based on international classification standards (ICD-11) for disease coding, significantly reducing misdiagnosis rates.

Medication conflict detection

Decision tree matching drug interaction rule library, LLM parsing non standardized expressions in prescriptions (such as product name and ingredient name mapping), and jointly outputting risk warnings and alternative solutions.

Personalized treatment recommendation

Integrating patient genomic data (decision tree rules) with clinical literature abstracts (LLM semantic analysis) to generate evidence-based personalized treatment plans.

06 Tree Search and Code Generation



Search path dynamic planning

- Heuristic path selection: Utilizing the semantic understanding ability of LLM to dynamically evaluate the potential value of search paths, combined with the logical branch weights of decision trees, prioritizing the expansion of high reward nodes. For example, in code generation tasks, probability models are used to predict the completion of different syntax structures, avoiding invalid searches.

- Real time pruning strategy: Based on the confidence threshold of the decision tree and the generation probability of LLM, it eliminates inefficient or contradictory branches in real time. For example, when there is a naming conflict or type mismatch in a variable, immediately terminate the current path and backtrack to a feasible solution.

- Resource allocation optimization: dynamically allocate computing resources based on task complexity, perform depth first search on critical paths (such as loop bodies or recursive calls), and adopt breadth first strategy for simple branches to balance efficiency and accuracy.

Collaborative exploration of code tree

Bidirectional knowledge transfer: LLM provides high-level semantic constraints (such as API call specifications), while decision trees are responsible for low-level logic validation (such as parameter type checking), and both correct errors through interactive feedback loops. For example, after LLM generates pseudocode, the decision tree converts it into a concrete syntax tree and provides feedback on missing exception handling.

Multi granularity node extension: In code tree traversal, LLM handles macro structures (such as function module partitioning) and decision trees refine micro logic (such as nested conditional judgments), forming hierarchical collaboration. For example, after LLM proposed the "sorting algorithm", the decision tree was expanded into the specific implementation steps of bubble sorting.

Collaborative exploration of code tree

01

Conflict resolution mechanism

When the generated results of LLM conflict with the rule base of the decision tree (such as overlapping variable scopes), a joint review process is initiated to prioritize retaining solutions that comply with compilation standards and explain the reasons for the conflict through natural language.

02

Incremental knowledge updating

Transforming new patterns (such as design patterns) generated by LLM into analyzable rules for decision trees, dynamically expanding the knowledge base. For example, encoding the implementation template of the observer pattern into a subset of rules for the decision tree.

Adaptive problem-solving strategy

Context aware adjustment

dynamically switching strategies based on user feedback, such as error logs or interaction history. For example, if there are multiple types of errors, strengthen the static check weights of the decision tree; If the logical complexity is high, increase the degree of freedom in generating LLM.

Hybrid reasoning mode

For deterministic tasks (such as mathematical operations), symbolic reasoning using decision trees is employed, while for open-ended tasks (such as natural language annotation generation), generative reasoning using LLM is enabled, and the results are integrated through weighted voting.

Failure backtracking learning

When a solution is rejected, record the failure path features and update the preference parameters of the joint model. For example, if users frequently reject lengthy code, optimize the simplicity reward function of LLM and the complexity threshold of decision trees.